Preparación del equipo

La instalación y configuración se repasan paso a paso durante el Día 1 (ejercicio guiado 1.G1). Si deseas adelantarla, verifica los siguientes puntos:

Lista de verificación previa

- ☐ Tengo un navegador moderno actualizado y sé abrir las herramientas de desarrollador con la tecla F12.
- ☐ Visual Studio Code está instalado (code.visualstudio.com) o lo instalaremos juntos el primer día.
- ☐ Node.js LTS está instalado: al escribir `node --version` en la terminal aparece un número de versión 18 o superior.
- ☐ He descargado y descomprimido la carpeta **Material_JS** que entrega el docente.
- ☐ Sé crear una carpeta nueva en un lugar fácil de encontrar y la nombro en minúsculas, sin espacios ni tildes.
- ☐ Tengo a mano papel y lápiz para planear la lógica antes de escribir código.

Material de apoyo: carpeta Material_JS

Junto con este documento se entrega una carpeta con los archivos de partida y los ejemplos de cada día. Los participantes trabajan con la carpeta `estudiantes/`; el docente dispone además de `docente/`, que contiene las soluciones de referencia, el servidor de práctica y las pruebas automáticas.

estructura de la carpeta

```
Material_JS/
├── README.md
├── estudiantes/
│   ├── package.json            (activa los módulos ES en Node)
│   ├── 00_plantilla_inicial/   (index.html, app.js, estilos.css)
│   ├── dia1/                   (ejemplos y ejercicio con errores)
│   ├── dia2/                   (funciones, arreglos y objetos)
│   ├── dia3/                   (DOM y eventos)
│   ├── dia4/                   (fetch, módulos y estados de la interfaz)
│   ├── dia5/                   (Node.js, API y proyecto_base/)
│   └── recursos/               (estilos-base.css, datos/ e img/)
└── docente/
    ├── servidor-datos.js       (API de práctica del Día 4)
    ├── solucionario/           (soluciones de referencia por día)
    └── pruebas/                (pruebas automáticas)
```

Regla de oro del trabajo

Una carpeta por proyecto. Guarda con **Ctrl + S** (o **Cmd + S** en Mac), recarga el navegador con **F5** y mantén la consola abierta (F12): los errores aparecen allí.

Escribe el código tú mismo en lugar de copiarlo: teclearlo ayuda a recordarlo y a detectar detalles.

Rúbrica de participación, organización del código y documentación (10 puntos)

Se evalúa durante toda la semana y se registra al cierre de cada día en la lista de seguimiento del docente.

Participación, organización y documentación (10 puntos)

Criterio	Excelente (100 %)	Satisfactorio (75 %)	Básico (50 %)	Insuficiente (0–25 %)	Puntos
Participación y actitud	Interviene, pregunta y ayuda a compañeros todos los días.	Participa en la mayoría de las actividades.	Participa solo cuando se le pide.	Casi no participa o interrumpe el trabajo.	**3**
Organización del código	Indentación uniforme, nombres claros, funciones pequeñas, archivos ordenados.	Código ordenado con fallos ocasionales de indentación o nombres.	Orden irregular; cuesta leer el código.	Código desordenado, nombres confusos o archivos dispersos.	**3**
Comentarios y documentación	Comentarios útiles y un README breve con lo que hace cada archivo.	Comentarios en las partes principales.	Comentarios escasos o poco claros.	Sin comentarios ni descripción del trabajo.	**2**
Puntualidad y entrega	Entrega todos los entregables diarios a tiempo.	Entrega casi todos a tiempo.	Entrega con retrasos o incompleto.	No entrega o entrega la minoría.	**2**
Total					**10**

Evaluación diagnóstica y evaluación final

La **evaluación diagnóstica** (al inicio del Día 1) y la **evaluación final de conocimientos** (al cierre del Día 5) son formativas: **no suman puntos** a los 100, pero permiten medir el avance de cada participante y del grupo comparando ambos resultados.

La nota de cada día se calcula con su rúbrica; el proyecto final aporta 35 puntos.

Evaluación diagnóstica

Responde de forma individual, sin buscar en Internet y sin preocuparte por el resultado: esta evaluación **no se califica**. Sirve para que el docente conozca el punto de partida del grupo y para que tú compares tus respuestas al finalizar el curso. Tiempo sugerido: 15 minutos (se aplica al inicio del Día 1).

1. En una página web, ¿qué lenguaje se usa normalmente para que la página reaccione a las acciones de la persona (clics, escritura, selecciones)?

- **A)** HTML
- **B)** CSS
- **C)** JavaScript
- **D)** Markdown

2. Una variable en programación es…

- **A)** Un archivo que se guarda en el disco duro
- **B)** Un nombre que identifica un valor guardado en la memoria
- **C)** Un error que el navegador muestra
- **D)** Una etiqueta de HTML

3. Un programa comienza con `x = 5` y después ejecuta `x = x + 2`. ¿Cuánto vale `x` al final?

- **A)** 5
- **B)** 2
- **C)** 7
- **D)** 52

4. Cuando abres una página web, ¿dónde se ejecuta el JavaScript que viene incluido en ella?

- **A)** En el navegador del usuario
- **B)** Únicamente en el servidor de la empresa
- **C)** En el router de la casa
- **D)** En la impresora

5. Java y JavaScript son el mismo lenguaje con dos nombres distintos.

( ) Verdadero ( ) Falso Justifica: _____

6. Si el código de una página tiene un error, el navegador siempre muestra un mensaje de error grande en la propia página.

( ) Verdadero ( ) Falso Justifica: _____

7. Para ejecutar JavaScript en el navegador es obligatorio instalar un compilador de pago.

( ) Verdadero ( ) Falso Justifica: _____

8. Describe con tus palabras una situación de un sitio web que uses (una tienda, una red social, un banco) en la que la página reaccione a lo que haces. ¿Qué crees que sucede «por detrás»?

Respuesta: _____

9. Escribe en lenguaje cotidiano, paso a paso, cómo calcularías el total de una compra si hay un descuento del 10 % cuando el valor supera los 100 000 pesos.

Respuesta: _____

10. ¿Qué esperas lograr al finalizar esta capacitación? Escribe una meta concreta.

Respuesta: _____

Día 1: Fundamentos de JavaScript

Objetivos de aprendizaje del día

Al terminar la jornada serás capaz de:

- **Explicar** qué es JavaScript, dónde se ejecuta (navegador y Node.js) y qué papel cumple ECMAScript.
- **Ejecutar** código JavaScript de dos maneras: con `node archivo.js` en la terminal y con `<script src="…" defer>` en una página que se abre con doble clic, consultando la consola de F12.
- **Declarar** variables con `const` y `let`, **distinguir** los tipos primitivos con `typeof`, **convertir** tipos y **comparar** valores con `===`.
- **Aplicar** operadores, plantillas de texto y estructuras condicionales (`if/else`, `switch`, ternario) para que un programa tome decisiones.
- **Construir** bucles (`for`, `while`, `for...of`) y funciones (declaradas y flecha) con parámetros y retorno, y **predecir** el ámbito en el que vive cada variable.
- **Interpretar** los mensajes de error de Node y del navegador y **depurar** un programa con la consola, la sentencia `debugger` y la pestaña Sources.

Lo que lograrás hoy: escribirás tus primeros programas de verdad: una ficha de presentación, un conversor de temperatura, el juego «Adivina el número» en la terminal, un contador de clics y una calculadora de propina que funciona en una página web. Hasta ahora has construido páginas que muestran información; hoy aprenderás a darles **comportamiento**: calcular, decidir y repetir. La Figura 1.1 muestra la plantilla inicial con la que empezarás a conectar JavaScript con una página, y la Figura 1.2 el aspecto de la calculadora de propina que construirás por la tarde.



Figura 1.1. Plantilla inicial del taller (00_plantilla_inicial) después de pulsar dos veces el botón.

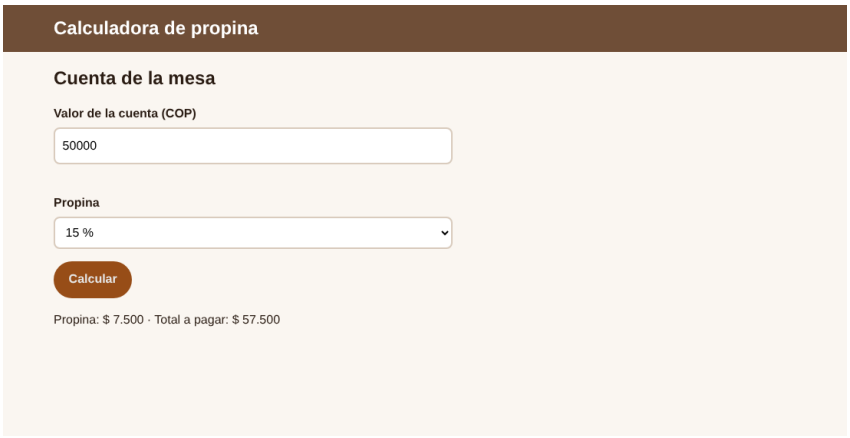


Figura 1.2. Calculadora de propina de la actividad 1.A5, con el resultado calculado por JavaScript.

Distribución de la jornada

Horario	Duración	Bloque	Qué se hace
8:00 – 8:20	20 min	**Repaso y activación**	Diagnóstico informal (qué has visto hacer a JavaScript), presentación del taller y comprobación de que Node.js, VS Code y el navegador funcionan.
8:20 – 9:40	80 min	**Explicación teórica**	Temas 1 a 19: qué es JavaScript, entorno, `console.log`, `<script>` y `defer`, variables, tipos, conversión, comparación, operadores, plantillas, condicionales, bucles, funciones, ámbito, entrada del usuario, errores y buenas prácticas.
9:40 – 10:00	20 min	**Demostraciones (parte 1)**	Ejemplos 1.1 (hola en Node y comentarios), 1.2 (hola en el navegador con consola F12 y un botón) y 1.3 (variables, tipos y conversión).
10:00 – 10:15	—	**DESCANSO**	Descanso no contabilizado
10:15 – 10:35	20 min	**Demostraciones (parte 2)**	Ejemplos 1.4 (operadores, plantillas y decisiones) y 1.5 (bucles, funciones y ámbito), prediciendo la salida antes de ejecutar.
10:35 – 11:45	70 min	**Práctica guiada**	Ejercicios guiados 1.G1 a 1.G4: ficha personal, conversor de temperatura, adivina el número con `readline` y contador de clics en la plantilla inicial.
11:45 – 12:00	15 min	**Ejercicios individuales (parte 1)**	Actividad 1.A1: explora la consola de F12 y el modo interactivo de Node.
12:00 – 13:00	—	**ALMUERZO**	Descanso no contabilizado
13:00 – 14:15	75 min	**Ejercicios individuales (parte 2)**	Actividades 1.A2 a 1.A6: laboratorio de tipos, tabla de multiplicar, validador de contraseña, propina en el navegador y detección de errores (ejercicio 1.6).

Horario	Duración	Bloque	Qué se hace
14:15 – 14:30	—	**DESCANSO**	Descanso no contabilizado
14:30 – 14:40	10 min	**Ejercicios individuales (parte 3)**	Actividad 1.A7: depurar el promedio de notas con Sources y `debugger` (ejercicio 1.7).
14:40 – 15:10	30 min	**Retos de profundización**	Retos 1.R1 y 1.R2 (el 1.R3 es opcional): adivina el número versión 2 y cuenta de Café Aroma por consola.
15:10 – 15:30	20 min	**Evaluación y cierre**	Preguntas de análisis, lista de verificación y revisión con la rúbrica de 10 puntos.

Tiempo efectivo de trabajo: 360 minutos (6 horas). Los descansos (15 + 60 + 15 minutos) no se cuentan.

Repaso y activación

Este es el primer día, así que no hay repaso de una jornada anterior: hay un **diagnóstico informal** para saber de dónde partimos. Responde en voz alta o en tu cuaderno:

1. **Lo que ya has visto:** has construido páginas con HTML y CSS. Piensa en una web que uses a diario (una tienda, un banco, una red social). ¿Qué cosas hace esa página que **no** podrías lograr solo con HTML y CSS? (por ejemplo, actualizar un total al agregar un producto sin recargar la página).
2. **Lo cotidiano:** cuando usas una hoja de cálculo o una calculadora, ¿qué haces cuando repites una operación muchas veces? ¿Y cuando el resultado depende de una condición («si el total supera 50.000, aplica descuento»)?
3. **Experiencia previa:** ¿has escrito alguna vez una instrucción para una máquina (una macro, una fórmula, un comando, un bloque de Scratch)? ¿Qué fue lo más difícil?
4. **Meta personal:** ¿qué te gustaría poder construir al terminar los cinco días? (el taller termina con una aplicación web completa: frontend dinámico más una API propia en Node.js).

Presentación del taller. Durante cinco días trabajarás así: el **Día 1** (hoy) aprende el lenguaje en la terminal y en la consola del navegador; el **Día 2** profundiza en funciones, arreglos y objetos; el **Día 3** conecta JavaScript con la página mediante el DOM y los eventos (Café Aroma interactivo); el **Día 4** consume datos de una API con `fetch`; y el **Día 5** crea el servidor con Node.js y reúne todo en un proyecto final. Cada día termina con un entregable que se evalúa con una rúbrica.

Comprobación del entorno (5 minutos). Antes de seguir, abre una terminal y escribe `node --version`. Debe aparecer un número de versión (el material se probó con Node 22; funciona con la versión 18 o superior). Si ves «node: command not found» o «"node" no se reconoce como un comando», avisa al instructor: se resuelve en un minuto instalando la versión LTS desde nodejs.org.

Explicación paso a paso

1. Qué es JavaScript y dónde se ejecuta

JavaScript es un lenguaje de programación: un conjunto de reglas para escribir instrucciones que una computadora puede ejecutar. Un **programa** es una lista de instrucciones que se ejecutan de arriba hacia abajo, una tras otra, y que pueden guardar datos, hacer cálculos, tomar decisiones y repetir tareas.

Analogía: imagina una casa. HTML es la estructura (paredes, puertas y ventanas), CSS es la decoración (pintura, cortinas, muebles) y JavaScript es la instalación eléctrica: los interruptores, los sensores y los timbres que hacen que algo **ocurra** cuando alguien interactúa con la casa.

Un programa en JavaScript necesita un **motor** (en inglés, engine) que lo lea y lo ejecute. Los navegadores incluyen uno: Chrome y Edge usan **V8**, Firefox usa SpiderMonkey y Safari usa JavaScriptCore. Node.js empaqueta el mismo motor V8 para que puedas ejecutar JavaScript **fuera** del navegador, directamente en tu computador o en un servidor. Por eso en este taller el lenguaje es el mismo, pero cambia lo que tienes a la mano en cada entorno:

Aspecto	Navegador	Node.js
Dónde corre	Dentro de una página web	En la terminal, sin página
Cómo se ejecuta	`<script src="app.js" defer>` y abrir el HTML	`node archivo.js`
Dónde ves `console.log`	Consola de las herramientas de desarrollo (F12)	La propia terminal
Qué ofrece	`document`, `window`, eventos, el DOM de la página	Archivos, red, `process`, servidores (`node:fs`, `node:http`)
Uso típico	Páginas interactivas (Días 3 y 4)	Scripts, herramientas y servidores (Día 5)

ECMAScript (abreviado ES) es el **estándar** que define cómo debe comportarse el lenguaje; JavaScript es su implementación más conocida. Cada año el comité TC39 publica una edición con novedades. La edición de 2015 (**ES2015**, también llamada ES6) fue un salto enorme: trajo `let`, `const`, las funciones flecha y las plantillas de texto. Cuando decimos «JavaScript moderno» nos referimos a lo que existe desde ES2015, y es lo único que usaremos en el taller.

> **No es Java**
> A pesar del nombre, JavaScript y Java son lenguajes distintos, con usos, reglas y herramientas diferentes. El nombre fue una decisión comercial de 1995. Si buscas ayuda en Internet, escribe siempre «JavaScript» y no solo «Java».

Pregunta de participación: si una página muestra un contador que sube cuando haces clic en un botón, ¿qué parte (HTML, CSS o JavaScript) hace que el número cambie? ¿Y qué parte define su color?

2. Preparar el entorno de trabajo

Necesitas cuatro herramientas. Todas son gratuitas y ya las usaste o las instalaste antes del taller:

Herramienta	Para qué la usas	Detalle
Visual Studio Code	Escribir el código	Abre la carpeta `Material_JS/estudiantes` con **Archivo > Abrir carpeta**.
Navegador moderno	Ver las páginas y la consola	Chrome, Edge o Firefox. Se abre la consola con **F12** (o `Ctrl + Shift + J` en Chrome y Edge).
Node.js (versión LTS)	Ejecutar JavaScript en la terminal	Comprueba la instalación con `node --version`. LTS significa Long Term Support: la versión estable recomendada.
Terminal	Dar órdenes a Node	Dentro de VS Code: menú **Terminal > Nuevo terminal**. Ya se abre en la carpeta del proyecto.

El flujo de trabajo del taller siempre es el mismo: **escribes** en VS Code, **guardas** con `Ctrl + S`, **ejecutas** (en la terminal o recargando la página con F5) y **observas** el resultado. Para ejecutar un archivo de la carpeta del día:

```
Terminal
node --version
cd dia1
node ejemplo-1.1-hola-node.js
```

Modo interactivo (REPL). Si escribes solo `node`, entras a un modo donde cada línea se ejecuta al instante y se muestra su resultado. Es ideal para probar ideas pequeñas. Se sale con `.exit` (o con `Ctrl + D`):

```
Sesión del modo interactivo de Node
> 2 + 3
5
> "Hola".length
4
> typeof 42
'number'
> const x = 10
undefined
> x * 2
20
```

En el navegador existe el equivalente: la pestaña **Console** de F12 también ejecuta lo que escribes. Las otras pestañas que usarás en el taller son **Elements** (el HTML y el CSS vivos de la página), **Sources** (el código, para depurar) y, desde el Día 4, **Network** (las peticiones de red).

> **El archivo package.json de la carpeta estudiantes**
>
> Dentro de `estudiantes` hay un `package.json` con la línea `"type": "module"`. Le indica a Node que los archivos `.js` son **módulos ES** y permite usar `import` (lo necesitarás hoy para `readline`). No lo borres ni lo muevas: todos los scripts de Node del taller dependen de él.

Cannot find module

Si al ejecutar `node archivo.js` aparece `Error: Cannot find module '…/archivo.js'`, casi siempre la terminal está en otra carpeta o el nombre está mal escrito.

Comprueba en qué carpeta estás (en VS Code, el nombre aparece en el prompt de la terminal), usa `cd dia1` para entrar a la carpeta correcta y escribe el nombre con ayuda de la tecla `Tab`, que lo autocompleta.

3. Tu primer programa: console.log y comentarios

La instrucción más usada del taller es `console.log(...)`: **muestra un valor** en la consola. Es tu herramienta para ver qué está haciendo el programa. Los **comentarios** son notas para las personas: JavaScript los ignora. Se escriben con `//` (una línea) o entre `/*` y `*/` (varias líneas).

ejemplo-1.1-hola-node.js

```
// ejemplo-1.1-hola-node.js
// Primer programa en Node.js. Ejecútalo desde la terminal con:
//   node ejemplo-1.1-hola-node.js

// Esto es un comentario de una línea: Node lo ignora por completo.

/*
  Esto es un comentario de varias líneas.
  Sirve para explicar bloques más largos de código.
*/

console.log("Hola, mundo");
console.log("Bienvenido al taller de JavaScript");

// console.log acepta varios valores separados por comas y los muestra con un espacio
entre ellos.
console.log("Resultado de 2 + 3 =", 2 + 3);
console.log("Versión de Node.js:", process.version);

// Otras formas de mostrar información en la consola
console.warn("Esto es una advertencia");
console.error("Esto es un mensaje de error (no detiene el programa)");

// Las instrucciones se ejecutan de arriba hacia abajo, una tras otra.
console.log("Fin del programa");
```

Salida en la terminal

```
Hola, mundo
Bienvenido al taller de JavaScript
Resultado de 2 + 3 = 5
Versión de Node.js: v22.22.0
Esto es una advertencia
Esto es un mensaje de error (no detiene el programa)
Fin del programa
```

- `console.log("Hola, mundo")` recibe un **texto entre comillas dobles** y lo imprime. Las instrucciones terminan con punto y coma (`;`).
- `console.log("Resultado de 2 + 3 =", 2 + 3)` recibe **dos valores separados por coma**: JavaScript calcula `2 + 3` y los muestra con un espacio entre ellos.
- `process.version` es una pieza que solo existe en Node.js: la versión instalada. La tuya puede ser distinta de la que se muestra aquí.

- `console.warn` y `console.error` se usan para advertencias y errores. En el navegador se muestran en amarillo y en rojo; en la terminal van por un canal aparte (stderr), pero el programa **continúa**.

> **Comillas, mayúsculas y orden**
> En este taller los textos van con **comillas dobles**. JavaScript distingue entre mayúsculas y minúsculas: `console.log` funciona, `Console.Log` o `console.Log` dan error. Y el código se ejecuta **de arriba hacia abajo**: la última línea del archivo se ejecuta al final.

Pregunta de participación: ¿qué crees que pasará si cambias el orden de las líneas `console.log("Hola, mundo")` y `console.log("Fin del programa")`? Pruébalo.

4. JavaScript en una página web: script, src y defer

Para que una página use JavaScript se enlaza un archivo `.js` con la etiqueta `<script>`, de forma parecida a como enlazas una hoja de estilos con `<link>`. Se escribe en el `<head>`, con dos atributos: `src` (la ruta del archivo) y `defer` (esperar a que el HTML esté listo).

ejemplo-1.2-hola-navegador.html (head)

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Hola desde el navegador</title>
  <link rel="stylesheet" href="../recursos/estilos-base.css">
  <!-- src: el código vive en un archivo aparte. defer: se ejecuta cuando el HTML ya está leído. -->
  <script src="ejemplo-1.2-hola-navegador.js" defer></script>
</head>
```

Por qué defer. El navegador lee el HTML de arriba hacia abajo. Si el script se ejecuta antes de que exista un botón, `document.getElementById("boton-saludo")` no lo encuentra y devuelve `null`. Con `defer` el archivo se descarga mientras se lee el HTML y **se ejecuta cuando la página ya está construida**. **Analogía:** el cocinero no empieza a preparar el pedido hasta que llegan todos los ingredientes.

Forma	Resultado	Recomendación
`<script src="app.js">` en `<head>` sin `defer`	Se ejecuta antes de que existan los elementos: `Cannot read properties of null`	Evitar
`<script src="app.js" defer>` en `<head>`	Se ejecuta con la página ya lista	**Usar siempre**
`<script src="app.js">` al final de `<body>`	Funciona, porque el HTML ya se leyó	Alternativa válida, pero es una convención antigua
`<script>` con código escrito dentro del HTML	Funciona, pero mezcla estructura y comportamiento	Solo para pruebas muy cortas

Ahora el archivo de JavaScript del ejemplo 1.2. Busca elementos de la página, cuenta los clics y escribe en la página y en la consola:

ejemplo-1.2-hola-navegador.js

```
// ejemplo-1.2-hola-navegador.js
// JavaScript en el navegador. Los mensajes de console.log aparecen en la consola de F12.

console.log("Hola desde el navegador");
console.log("Título de la página:", document.title);

// Buscamos dos elementos de la página por su id.
const botonSaludo = document.getElementById("boton-saludo");
const mensaje = document.getElementById("mensaje");

// Un dato que cambia con cada clic.
let clics = 0;

// addEventListener: "cuando ocurra este evento, ejecuta esta función".
botonSaludo.addEventListener("click", () => {
  clics = clics + 1;
  console.log(`Clic número ${clics}`);
  mensaje.textContent = `Has hecho clic ${clics} vez(es).`;
});
```

Consola de F12 después de tres clics

```
Hola desde el navegador
Título de la página: Hola desde el navegador
Clic número 1
Clic número 2
Clic número 3
```

Lo que se ve en la página: el texto cambia a «Has hecho clic 3 vez(es).» (Figura 1.3). Las tres líneas con `getElementById`, `addEventListener` y `textContent` son un **adelanto**: el Día 3 las estudiaremos a fondo. Por hoy basta con entender que `addEventListener("click", función)` significa «cuando ocurra un clic, ejecuta esta función».



Figura 1.3. Ejemplo 1.2 después de tres clics en el botón «Saludar».

Dos modos de ejecución. Los scripts de Node se ejecutan con `node archivo.js`. Las páginas del Día 1 se abren con **doble clic** sobre el archivo `.html` (la dirección del navegador empieza con `file:///`), y por eso usan scripts clásicos, sin módulos ni `fetch`. Esos dos recursos necesitan un servidor local y los usaremos desde el Día 4.

La plantilla inicial. En `estudiantes/00_plantilla_inicial/` tienes un proyecto mínimo listo para usar: `index.html`, `app.js`, `estilos.css` y la carpeta `img/`. Cada vez que empieces una página nueva, **copia esa carpeta** dentro de `estudiantes` (al lado de la original, para que la ruta `../recursos/estilos-base.css` siga funcionando) y cámbiale el nombre.

00_plantilla_inicial/app.js

```
// app.js · Plantilla inicial del taller de JavaScript.
// Este archivo se carga con <script src="app.js" defer> al final del <head> de
index.html.

console.log("app.js cargado: la página está lista");

// 1. Buscamos los elementos de la página que necesitamos.
const botonProbar = document.getElementById("boton-probar");
const mensaje = document.getElementById("mensaje");

// 2. Guardamos un dato que cambia con el tiempo.
let veces = 0;

// 3. Cuando alguien pulsa el botón, ejecutamos esta función.
botonProbar.addEventListener("click", () => {
  veces = veces + 1;
  console.log(`Botón pulsado ${veces} vez(es)`);
  mensaje.textContent = `JavaScript funciona: has pulsado el botón ${veces} vez(es).`;
});
```

prompt y alert: solo una curiosidad

El ejemplo 1.2 incluye un botón que usa `prompt` (pide un texto en una ventana) y `alert` (muestra un aviso). Existen desde los inicios de la web, pero **bloquean la página**, no se pueden diseñar y no son accesibles para todas las personas. En el taller usaremos `console.log` y elementos de la propia página; en la terminal, `readline` (tema 17).

Cannot read properties of null

Si la consola muestra `TypeError: Cannot read properties of null (reading 'addEventListener')`, el script no encontró el elemento. Revisa tres cosas: que el `<script>` tenga `defer`, que el `id` del HTML y el del JavaScript sean idénticos (incluidas las mayúsculas) y que el archivo `.js` esté guardado.

Pregunta de participación: abre la plantilla inicial, quita `defer` de la etiqueta `<script>`, guarda y recarga con F5. ¿Qué ves en la consola? Vuelve a ponerlo.

5. Variables: let y const

Una **variable** es un espacio con nombre en la memoria donde guardas un dato para usarlo después. **Analogía:** una caja con una etiqueta; escribes el nombre en la etiqueta y metes algo dentro. Con `const` (constante) la caja queda sellada: el nombre siempre apunta al mismo valor. Con `let` la caja se puede abrir y cambiar su contenido.

ejemplo-1.3-variables-tipos-y-conversion.js (1. variables)

```
// ---------- 1. Variables: const y let ----------
const nombreCafeteria = "Café Aroma"; // const: el valor no se puede reasignar
let mesasOcupadas = 3;                // let: el valor puede cambiar

console.log(nombreCafeteria, "tiene", mesasOcupadas, "mesas ocupadas");
mesasOcupadas = mesasOcupadas + 2;    // reasignar es válido con let
console.log("Ahora hay", mesasOcupadas, "mesas ocupadas");

// Reasignar una constante provoca un error. Usamos try/catch (se estudia el Día
2)
// solo para mostrar el mensaje sin detener el programa.
try {
  nombreCafeteria = "Otro nombre";
} catch (error) {
  console.log("Error al reasignar una constante:", error.message);
}
```

Salida en la terminal

```
Café Aroma tiene 3 mesas ocupadas
Ahora hay 5 mesas ocupadas
Error al reasignar una constante: Assignment to constant variable.
```

Observa la línea `mesasOcupadas = mesasOcupadas + 2;`: **primero** se calcula el lado derecho (3 + 2) y **después** el resultado se guarda en la variable. El signo `=` en programación no significa «es igual a»: significa «guarda este valor». La reasignación de la constante provoca `TypeError: Assignment to constant variable.`; el bloque `try/catch` solo sirve aquí para mostrar el mensaje sin detener el programa (lo estudiarás el Día 2).

Reglas y convenciones para los nombres:

- Pueden contener letras, dígitos, `_` y `$`, pero **no pueden empezar con un dígito** ni coincidir con una palabra reservada (`const`, `let`, `if`, `for`, `function`, `return`…).
- Distinguen mayúsculas de minúsculas: `total` y `Total` son variables diferentes.
- En el taller se escriben **en español, sin tildes ni ñ y en camelCase**: `precioUnitario`, `mesasOcupadas`, `anioNacimiento` (no `añoNacimiento`).
- Deben describir el contenido: `tasaIva` dice más que `t` o `x1`. Los nombres que empiezan con verbo se reservan para funciones (`calcularTotal`).

	const	let
¿Se puede reasignar?	No: da `TypeError`	Sí
¿Hay que darle valor al declararla?	Sí, obligatorio	No (vale `undefined` hasta que se asigne)
Ámbito	De bloque	De bloque
¿Cuándo usarla?	**Por defecto**, siempre que el valor no cambie	Solo cuando el valor va a cambiar (contadores, acumuladores)

> **const por defecto**
> Escribe `const` siempre y cámbialo a `let` solo cuando el programa te lo exija. Así, quien lea tu código sabe qué valores son fijos y qué valores cambian, y el lenguaje te avisa si reasignas por error algo que no debía cambiar.

Pregunta de participación: ¿cuál de estas variables debería ser `const` y cuál `let`? El nombre de la cafetería, el número de clientes atendidos hoy, el porcentaje de IVA.

6. Por qué evitar var

Antes de 2015 la única forma de declarar variables era `var`. Sigue funcionando, y la verás en tutoriales antiguos, pero tiene comportamientos que producen errores difíciles de encontrar. Por eso en este taller **nunca** la usamos; el ejemplo siguiente es la única excepción y existe para que veas por qué:

ejemplo-1.3-variables-tipos-y-conversion.js (2. var)

```
// ---------- 2. Por qué evitar var ----------
function demostrarVar() {
  console.log("Antes de declararla, var vale:", elevada); // undefined, no da
error
  var elevada = 10;

  if (true) {
    var conVar = "var se ve fuera del bloque";
    let conLet = "let solo existe dentro del bloque";
    console.log(conLet);
  }
  console.log(conVar);
  console.log("Tipo de conLet fuera del bloque:", typeof conLet);
}
demostrarVar();
```

Salida en la terminal

```
Antes de declararla, var vale: undefined
let solo existe dentro del bloque
var se ve fuera del bloque
Tipo de conLet fuera del bloque: undefined
```

- **Se «eleva» (hoisting):** puedes usar una variable `var` antes de declararla y obtienes `undefined` en lugar de un error. Con `let` y `const` ocurre lo correcto: error claro.
- **Ignora los bloques:** `conVar` declarada dentro del `if` sigue viva fuera de las llaves; `conLet` no. Una variable que «se escapa» puede ser modificada sin querer desde otro lugar.
- **Contamina la página:** en un script clásico del navegador, una `var` declarada al nivel del archivo se convierte en propiedad de `window` (`window.viejo` vale 1), mientras que `let` y `const` no (`window.nuevo` es `undefined`). Así dos scripts pueden pisarse sin avisar.

Pregunta de participación: si ves un `var` en un código que encontraste en Internet, ¿por cuál palabra lo reemplazarías por defecto?

7. Tipos primitivos y typeof

Cada valor tiene un **tipo** que determina qué se puede hacer con él (no puedes «multiplicar» dos nombres, pero sí dos números). JavaScript tiene cinco tipos primitivos básicos que usarás todos los días y que el operador `typeof` te permite consultar:

Tipo	Qué representa	Ejemplos	typeof
`string`	Texto	`"Espresso"`, `""`, `"123"`	`"string"`
`number`	Números enteros y decimales (con punto), más `NaN` e `Infinity`	`6500`, `0.1`, `-3`	`"number"`
`boolean`	Verdadero o falso	`true`, `false`	`"boolean"`
`null`	Ausencia de valor **asignada a propósito**	`null`	`"object"` (rareza histórica)
`undefined`	Variable que existe pero **todavía no tiene valor**	`let x;`	`"undefined"`

Existen otros dos primitivos, `bigint` (enteros enormes) y `symbol` (identificadores únicos), que no necesitarás en este taller.

ejemplo-1.3-variables-tipos-y-conversion.js (3. tipos)

```
// ---------- 3. Tipos primitivos y typeof ----------
const texto = "Espresso";     // string
const precio = 6500;          // number (enteros y decimales son del mismo tipo)
const descuento = 0.1;        // number
const disponible = true;      // boolean
const sinValor = null;        // null: ausencia de valor puesta a propósito
let sinAsignar;               // undefined: la variable existe pero no tiene valor

console.log("texto:", typeof texto);
console.log("precio:", typeof precio);
console.log("descuento:", typeof descuento);
console.log("disponible:", typeof disponible);
console.log("sinValor:", typeof sinValor);       // "object": un error histórico del
lenguaje
console.log("sinAsignar:", typeof sinAsignar);

// Particularidades de number
console.log(0.1 + 0.2);                          // 0.30000000000000004
console.log((0.1 + 0.2).toFixed(2));             // "0.30"
console.log(10 / 0, -10 / 0);                    // Infinity -Infinity
console.log(typeof NaN);                         // "number": NaN significa "no es un
número"
```

Salida en la terminal

```
texto: string
precio: number
descuento: number
disponible: boolean
sinValor: object
sinAsignar: undefined
0.30000000000000004
0.30
Infinity -Infinity
number
```

Cuidado con los decimales y con NaN

`0.1 + 0.2` da `0.30000000000000004`. No es un fallo de JavaScript: los computadores guardan los decimales en binario y algunos no se representan con exactitud. Para mostrar un resultado, redondea con `toFixed(2)` (devuelve un texto) o formatea con `Intl.NumberFormat`.

`NaN` («Not a Number») es el resultado de una operación numérica imposible, como `"hola" * 2`. Curiosamente su `typeof` es `"number"`.

Pregunta de participación: ¿de qué tipo es `"2026"`? ¿Y `2026`? ¿Y `true`? Compruébalo en la consola con `typeof`.

8. Conversión de tipos

Muchas veces un dato llega con un tipo y lo necesitas con otro. Lo más frecuente: **todo lo que escribe una persona** (en un campo de formulario o en la terminal) llega como **texto**, aunque sean dígitos. Si no lo conviertes, `"25" + 5` da `"255"` y no `30`.

ejemplo-1.3-variables-tipos-y-conversion.js (4. conversión)

```
// ---------- 4. Conversión de tipos ----------
const entrada = "25"; // todo lo que llega de un campo de texto o de la terminal es string

console.log(entrada + 5);                          // "255": con + gana la concatenación
console.log(Number(entrada) + 5);                  // 30
console.log("10" - 4, "10" * "2");                 // 6 20: -, * y / convierten a número
console.log(Number("12abc"));                      // NaN
console.log(parseInt("42px"), parseFloat("3.14 metros")); // 42 3.14
console.log(String(123) + 1);                      // "1231"
console.log(Number.isNaN(Number("abc")));          // true: la forma correcta de detectar
NaN
```

Salida en la terminal

```
255
30
6 20
NaN
42 3.14
1231
true
```

Función	Qué hace	Ejemplo y resultado
`Number(x)`	Convierte a número; si no puede, da `NaN`	`Number("25")` → `25`; `Number("12abc")` → `NaN`; `Number("")` → `0`
`parseInt(x)`	Lee un entero al inicio del texto	`parseInt("42px")` → `42`
`parseFloat(x)`	Lee un decimal al inicio del texto	`parseFloat("3.14 metros")` → `3.14`
`String(x)`	Convierte a texto	`String(123)` → `"123"`
`Boolean(x)`	Convierte a verdadero o falso (ver tema 13)	`Boolean("")` → `false`
`Number.isNaN(x)`	Comprueba si el valor es `NaN`	`Number.isNaN(Number("abc"))` → `true`

Conversión implícita. JavaScript también convierte por su cuenta, según el operador: `+` **concatena** si alguno de los lados es texto, pero `-`, `*` y `/` siempre convierten a número. Por eso `"10" - 4` da `6` y `"5" + 3` da `"53"`. Prefiere la conversión **explícita** (`Number(...)`): deja claro lo que pretendes.

> **Number vacío y NaN silencioso**
>
> `Number("")` y `Number("  ")` dan `0`, no `NaN`: una persona que pulsa Enter sin escribir nada se convierte en un «0». Antes de convertir, comprueba que el texto no esté vacío con `texto.trim() === ""`.
>
> `NaN` no lanza ningún error: se propaga por los cálculos y aparece al final como «Total: NaN». Si ves `NaN`, busca el dato que no se convirtió bien.

Pregunta de participación: ¿qué resultado esperas de `"3" * "4"`? ¿Y de `"3" + "4"`? ¿Por qué cambia el comportamiento con el mismo par de valores?

9. Comparación: == frente a ===

Para comparar valores existen dos familias de operadores. La **igualdad estricta** `===` (y su contraria `!==`) compara **valor y tipo**, sin convertir nada. La **igualdad laxa** `==` (y `!=`) intenta convertir los tipos antes de comparar, lo que produce resultados sorprendentes:

ejemplo-1.3-variables-tipos-y-conversion.js (5. comparación)

```
// ---------- 5. Comparación: == frente a === ----------
console.log(5 == "5", 5 === "5");                 // true false
console.log(0 == false, 0 === false);             // true false
console.log("" == 0, "" === 0);                   // true false
console.log(null == undefined, null === undefined); // true false
console.log(NaN === NaN);                         // false: NaN no es igual ni a
sí mismo
console.log(5 !== "5");                           // true
```

Salida en la terminal

```
true false
true false
true false
true false
false
true
```

`0 == false` y `"" == 0` dan `true`, aunque son cosas distintas. Con `===` esas comparaciones dan `false`, que es lo que casi siempre se espera. Dos casos aparte: `null == undefined` es `true` pero `null === undefined` es `false`, y **NaN no es igual a nada, ni siquiera a sí mismo** (usa `Number.isNaN`).

Operador	Significado	Ejemplo
`===`	Igual en valor y tipo	`5 === 5` → `true`; `5 === "5"` → `false`
`!==`	Distinto en valor o tipo	`5 !== "5"` → `true`
`< >`	Menor que, mayor que	`3 < 7` → `true`
`<= >=`	Menor o igual, mayor o igual	`7 >= 7` → `true`

> **Regla del taller**
> Usa **siempre** `===` y `!==`. Si necesitas comparar un texto con un número, convierte primero con `Number(...)` y luego compara. Verás que esta regla evita uno de los errores más frecuentes: leer `"5"` de un campo y compararlo con el número `5`.

Pregunta de participación: sin ejecutar, ¿qué da `0 === false`? ¿Y `"1" == 1`? ¿Y `"1" === 1`?

10. Operadores aritméticos, de asignación y lógicos

Un **operador** es un símbolo que combina valores para producir otro valor. Los aritméticos hacen cálculos; los de asignación guardan resultados en variables; los de comparación y los lógicos producen `true` o `false`.

ejemplo-1.4-operadores-y-decisiones.js (operadores)

```
// ---------- 1. Operadores aritméticos y de asignación ----------
const a = 17;
const b = 5;
console.log("suma:", a + b);                // 22
console.log("resta:", a - b);               // 12
console.log("multiplicación:", a * b);      // 85
console.log("división:", a / b);            // 3.4
console.log("residuo:", a % b);             // 2
console.log("potencia:", a ** 2);           // 289
console.log("división entera:", Math.floor(a / b)); // 3

let saldo = 100;
saldo += 20;   // saldo = saldo + 20  -> 120
saldo -= 10;   // saldo = saldo - 10  -> 110
saldo *= 2;    // saldo = saldo * 2   -> 220
saldo++;       // saldo = saldo + 1   -> 221
console.log("saldo final:", saldo);

// ---------- 2. Operadores de comparación y lógicos ----------
const edad = 20;
const tieneEntrada = true;
console.log(edad >= 18 && tieneEntrada);    // true: ambas condiciones se
cumplen
console.log(edad < 18 || tieneEntrada);     // true: basta con que una se cumpla
console.log(!tieneEntrada);                 // false: invierte el valor

// Valores por defecto: || usa el de la derecha si el de la izquierda es falsy;
// ?? lo usa solo si el de la izquierda es null o undefined.
const nombreEscrito = "";
console.log("con ||:", nombreEscrito || "Invitado");   // Invitado
console.log("con ??:", `"${nombreEscrito ?? "Invitado"}"`);   // "" (cadena
vacía)
```

```
Salida en la terminal
suma: 22
resta: 12
multiplicación: 85
división: 3.4
residuo: 2
potencia: 289
división entera: 3
saldo final: 221
true
true
false
con ||: Invitado
con ??: ""
```

Operador	Nombre	Ejemplo	Resultado
`+ - * /`	Suma, resta, multiplicación, división	`17 / 5`	`3.4`
`%`	Residuo (resto de la división)	`17 % 5`	`2`
`**`	Potencia	`17 ** 2`	`289`
`+= -= *= /=`	Asignación compuesta	`saldo += 20`	`saldo = saldo + 20`
`++ --`	Sumar o restar 1	`saldo++`	`saldo = saldo + 1`
`&&`	Y: ambas condiciones	`edad >= 18 && tieneEntrada`	`true` solo si las dos son `true`
`||`	O: al menos una condición	`edad < 18 || tieneEntrada`	`true` si alguna es `true`
`!`	No: invierte	`!tieneEntrada`	`false`

El residuo es más útil de lo que parece: `numero % 2 === 0` indica que un número es par, y `numero % 15 === 0` que es múltiplo de 15. **Orden de evaluación:** primero `**`, luego `*`, `/` y `%`, después `+` y `-`, luego las comparaciones, `&&` y por último `||`. Ante la duda, usa **paréntesis**: `(subtotal + impuesto) / personas` no es lo mismo que `subtotal + impuesto / personas`.

Valores por defecto con || y ??. `a || b` devuelve `b` si `a` es falsy (incluye cadena vacía y `0`); `a ?? b` devuelve `b` solo si `a` es `null` o `undefined`. En el ejemplo, la cadena vacía hace que `||` elija «Invitado» y que `??` conserve la cadena vacía. Elige según lo que quieras considerar «sin valor».

Pregunta de participación: si `saldo` vale 221, ¿qué valor tendrá después de `saldo %= 10`? ¿Y de `saldo **= 2`?

11. Template literals: texto con variables

Unir textos con `+` se vuelve incómodo cuando hay muchas variables. Los **template literals** (plantillas de texto) se escriben entre **comillas invertidas** (el acento grave) y permiten insertar el valor de cualquier expresión con `${...}`. También admiten varias líneas.

ejemplo-1.4-operadores-y-decisiones.js (plantillas)

```
// ---------- 3. Template literals ----------
const producto = "Capuchino";
const precioUnitario = 9500;
const cantidad = 3;
const total = precioUnitario * cantidad;

console.log("Pediste " + cantidad + " " + producto + " y pagas $" + total);    // forma
antigua
console.log(`Pediste ${cantidad} ${producto} y pagas $${total}`);               // con
plantilla
console.log(`Con propina del 10 %: $${total + total * 0.1}`);

const resumen = `Resumen del pedido
  Producto: ${producto}
  Cantidad: ${cantidad}
  Total:    ${total}`;
console.log(resumen);

const formatoCOP = new Intl.NumberFormat("es-CO", { style: "currency", currency: "COP",
maximumFractionDigits: 0 });
console.log("Total con formato:", formatoCOP.format(total));
```

Salida en la terminal

```
Pediste 3 Capuchino y pagas $28500
Pediste 3 Capuchino y pagas $28500
Con propina del 10 %: $31350
Resumen del pedido
  Producto: Capuchino
  Cantidad: 3
  Total:    28500
Total con formato: $ 28.500
```

- En `${cantidad}` se inserta el valor de la variable; en `${total * 1.1}` se evalúa una operación completa.
- `$${total}` produce el símbolo `$` seguido del valor: el primer `$` es texto y el segundo abre la expresión.
- El texto de varias líneas conserva los saltos y las sangrías que escribas dentro de las comillas invertidas.
- Para mostrar dinero con puntos de mil, `Intl.NumberFormat("es-CO", …)` (que verás con detalle el Día 2) formatea el número como pesos colombianos.

> **Plantillas siempre**
> En el taller usamos plantillas para construir **todos** los mensajes con variables. Son más legibles que la concatenación con `+` y evitan errores de espacios olvidados.

Pregunta de participación: ¿qué mostraría una plantilla cuyo contenido es Total: ${2 + 3}? ¿Y otra cuyo contenido es simplemente Total: 2 + 3, sin el signo de dólar ni las llaves?

12. Condicionales: if/else, switch y ternario

Un **condicional** ejecuta un bloque solo si se cumple una condición. **Analogía:** un semáforo: si está en verde, avanzas; si está en amarillo, te preparas; en cualquier otro caso, te detienes. La condición va entre paréntesis y es cualquier expresión que se pueda interpretar como verdadera o falsa.

ejemplo-1.4-operadores-y-decisiones.js (decisiones)

```
// ---------- 4. if / else if / else ----------
const hora = 15;
let saludo;

if (hora < 12) {
  saludo = "Buenos días";
} else if (hora < 18) {
  saludo = "Buenas tardes";
} else {
  saludo = "Buenas noches";
}
console.log(`${saludo}, bienvenido a Café Aroma`);

// ---------- 5. switch ----------
const categoria = "postres";

switch (categoria) {
  case "bebidas-calientes":
    console.log("Sección: bebidas calientes");
    break;
  case "postres":
  case "panaderia":
    console.log("Sección: dulces y panadería");
    break;
  default:
    console.log("Categoría desconocida");
}

// ---------- 6. Operador ternario ----------
const puntos = 8;
const estado = puntos >= 5 ? "aprobado" : "reprobado";
console.log(`Con ${puntos} puntos: ${estado}`);
```

Salida en la terminal

```
Buenas tardes, bienvenido a Café Aroma
Sección: dulces y panadería
Con 8 puntos: aprobado
```

Estructura	Cuándo usarla	Forma
`if / else if / else`	Decisiones con rangos o condiciones distintas	`if (cond) { … } else if (cond) { … } else { … }`
`switch`	Comparar **un mismo valor** contra varios casos exactos	`switch (valor) { case "a": …; break; default: … }`
Ternario `? :`	Elegir **un valor** entre dos opciones, en una línea	`const estado = puntos >= 5 ? "aprobado" : "reprobado";`

- En `if/else if`, JavaScript evalúa de arriba hacia abajo y ejecuta **solo la primera rama** que se cumple. Por eso `hora < 12` va antes que `hora < 18`.
- En `switch`, cada `case` termina con `break`; sin él, la ejecución «cae» al siguiente caso. Dos `case` seguidos (`"postres"` y `"panaderia"`) comparten el mismo bloque.
- `default` se ejecuta cuando ningún caso coincide: úsalo para valores inesperados.

- El ternario es cómodo para **asignar** un valor; si necesitas ejecutar varias instrucciones, usa `if`.

> **Un solo signo igual en la condición**
> Escribir `if (total = 0)` en lugar de `if (total === 0)` **asigna** 0 a la variable y la condición queda falsa. No da error: el programa simplemente se comporta mal. Es uno de los errores del ejercicio 1.6. Pon siempre llaves y revisa que la condición lleve `===`.

Pregunta de participación: si `hora` vale 12, ¿qué saludo muestra el ejemplo? ¿Y si `hora` vale 18?

13. Valores truthy y falsy

Dentro de un `if` JavaScript convierte cualquier valor a verdadero o falso. Los valores que se convierten en falso son pocos y conviene memorizarlos: **false, 0, "" (texto vacío), null, undefined y NaN**. Todo lo demás es truthy (se comporta como verdadero), incluidos `"0"`, `"false"`, `[]` y `{}`.

ejemplo-1.4-operadores-y-decisiones.js (truthy y falsy)

```
// ---------- 7. Valores truthy y falsy ----------
console.log("false     ->", Boolean(false) ? "truthy" : "falsy");
console.log("0         ->", Boolean(0) ? "truthy" : "falsy");
console.log(`""         ->`, Boolean("") ? "truthy" : "falsy");
console.log("null      ->", Boolean(null) ? "truthy" : "falsy");
console.log("undefined ->", Boolean(undefined) ? "truthy" : "falsy");
console.log("NaN       ->", Boolean(NaN) ? "truthy" : "falsy");
console.log(`"0"        ->`, Boolean("0") ? "truthy" : "falsy");
console.log(`"hola"     ->`, Boolean("hola") ? "truthy" : "falsy");
console.log("[]        ->", Boolean([]) ? "truthy" : "falsy");
```

Salida en la terminal

```
false     -> falsy
0         -> falsy
""        -> falsy
null      -> falsy
undefined -> falsy
NaN       -> falsy
"0"       -> truthy
"hola"    -> truthy
[]        -> truthy
```

Esto permite escribir comprobaciones breves: `if (nombre) { … }` se cumple solo si `nombre` tiene contenido. Pero ten cuidado: `if (cantidad)` también es falso cuando `cantidad` vale `0`, que puede ser un dato legítimo. Cuando importe la diferencia, compara de forma explícita (`cantidad !== undefined`).

Pregunta de participación: una persona escribe «0» en un campo de texto. ¿`if (respuesta)` se cumple? ¿Y si antes conviertes con `Number(respuesta)`?

14. Bucles: for, while y for...of

Un **bucle** repite un bloque de instrucciones. **Analogía:** una lavadora que repite el ciclo de enjuague hasta que se cumple una condición. Cada repetición se llama **iteración** o vuelta.

ejemplo-1.5-bucles-y-funciones.js (bucles)

```
// ---------- 1. Bucle for ----------
// for (inicio; condición; paso): se repite mientras la condición sea verdadera.
for (let i = 1; i <= 5; i++) {
```

```
  console.log(`Vuelta número ${i}`);
}

// ---------- 2. Bucle while ----------
// Se usa cuando no sabemos cuántas vueltas harán falta.
let ahorro = 100;
let meses = 0;

while (ahorro < 1000) {
  ahorro = ahorro * 2;
  meses++;
}
console.log(`Se supera 1000 después de ${meses} meses (ahorro: ${ahorro})`);

// ---------- 3. break y continue ----------
for (let n = 1; n <= 10; n++) {
  if (n % 2 === 0) {
    continue;   // salta a la siguiente vuelta
  }
  if (n > 7) {
    break;      // termina el bucle
  }
  console.log(`Número impar: ${n}`);
}

// ---------- 4. for...of: recorrer una lista ----------
// Un arreglo (array) es una lista ordenada de valores. El Día 2 los estudia a
fondo.
const bebidas = ["Espresso", "Capuchino", "Latte"];
console.log("Cantidad de bebidas:", bebidas.length);
console.log("Primera bebida:", bebidas[0]);

for (const bebida of bebidas) {
  console.log(`- ${bebida}`);
}

for (const letra of "Café") {
  console.log(letra);
}
```

Salida en la terminal

```
Vuelta número 1
Vuelta número 2
Vuelta número 3
Vuelta número 4
Vuelta número 5
Se supera 1000 después de 4 meses (ahorro: 1600)
Número impar: 1
Número impar: 3
Número impar: 5
Número impar: 7
Cantidad de bebidas: 3
Primera bebida: Espresso
- Espresso
- Capuchino
- Latte
C
a
f
é
```

Bucle	Cuándo usarlo	Partes
`for`	Sabes cuántas vueltas dar	`for (inicio; condición; paso)`: la variable de control se crea, se comprueba antes de cada vuelta y se actualiza al final
`while`	No sabes cuántas vueltas harán falta: repites **mientras** se cumpla algo	`while (condición) { … }`: dentro del bloque algo debe cambiar la condición
`for...of`	Recorrer los elementos de una lista (arreglo) o los caracteres de un texto	`for (const elemento of lista) { … }`

- `break` termina el bucle por completo; `continue` salta a la siguiente vuelta. En el ejemplo, `continue` ignora los pares y `break` detiene el bucle cuando `n` supera 7.
- Un **arreglo** (en inglés, array) es una lista ordenada de valores entre corchetes: `["Espresso", "Capuchino", "Latte"]`. Se accede a cada elemento por su posición, que empieza en **0** (`bebidas[0]`), y `bebidas.length` dice cuántos hay. Mañana los estudiaremos a fondo; hoy solo los usamos para recorrerlos.
- En `for...of` declara el elemento con `const`: en cada vuelta nace una constante nueva.

> **El bucle infinito**
> Si la condición nunca se vuelve falsa (por ejemplo, un `while` donde olvidas actualizar la variable), el programa no termina. En la terminal se detiene con `Ctrl + C`; en el navegador la pestaña se congela y hay que cerrarla. Antes de ejecutar un `while`, pregúntate: **¿qué instrucción hará que la condición deje de cumplirse?**

Pregunta de participación: ¿cuántas veces se muestra «Vuelta» en `for (let i = 0; i < 5; i++)`? ¿Y si cambias `<` por `<=`? Este tipo de diferencia de una unidad es un clásico llamado error por uno.

15. Funciones: declaración, parámetros y retorno

Una **función** es un bloque de código con nombre que puedes ejecutar (**llamar**) las veces que quieras. **Analogía:** una receta: tiene un nombre, unos ingredientes de entrada (**parámetros**) y produce un plato (**valor de retorno**). Cada vez que la pides con ingredientes distintos, obtienes un resultado distinto.

ejemplo-1.5-bucles-y-funciones.js (funciones)

```
// ---------- 5. Funciones: declaración, parámetros y retorno ----------
function calcularTotal(precio, cantidad) {
  const total = precio * cantidad;
  return total; // devuelve el resultado a quien llamó a la función
}

console.log("Total:", calcularTotal(9500, 3));       // 28500

function saludar(nombre) {
  console.log(`Hola, ${nombre}`);                    // hace algo, pero no
devuelve nada
}
```

```
const resultado = saludar("Camila");
console.log("saludar devolvió:", resultado);          // undefined

function esMayorDeEdad(edad) {
  return edad >= 18;                                  // devuelve true o false
}
console.log(esMayorDeEdad(20), esMayorDeEdad(15));    // true false

// ---------- 6. Funciones flecha (arrow functions) ----------
const calcularIva = (valor) => valor * 0.19;
const sumar = (x, y) => x + y;
const describirPrecio = (nombre, precio) => {
  const texto = `${nombre} cuesta $${precio}`;
  return texto;
};

console.log(calcularIva(10000));                      // 1900
console.log(sumar(2, 3));                             // 5
console.log(describirPrecio("Espresso", 6500));
```

Salida en la terminal

```
Total: 28500
Hola, Camila
saludar devolvió: undefined
true false
1900
5
Espresso cuesta $6500
```

- **Declaración:** `function calcularTotal(precio, cantidad) { … }`. `precio` y `cantidad` son los **parámetros** (nombres internos); al llamar `calcularTotal(9500, 3)`, los valores 9500 y 3 son los **argumentos**.
- `return` entrega un valor a quien llamó y termina la función. Sin `return`, la función devuelve `undefined`, como se ve en `saludar`: hace algo (imprime) pero no entrega nada.
- **Función flecha** (arrow function): una forma más corta que se escribe `(parámetros) => resultado`. Si el cuerpo es una sola expresión, el `return` es implícito; si usas llaves, hay que escribir `return`. Mañana veremos con más calma por qué se usan tanto.
- Dale a cada función **una sola tarea** y un nombre que empiece con verbo: `calcularIva`, `esMayorDeEdad`, `describirPrecio`.

	Declaración	Flecha
Sintaxis	`function sumar(x, y) { return x + y; }`	`const sumar = (x, y) => x + y;`
Se puede llamar antes de la línea donde se escribe	Sí	No (es una constante)
Uso en el taller	Funciones principales, con nombre	Funciones cortas y callbacks (Días 2 y 3)

Pregunta de participación: ¿qué devuelve `esMayorDeEdad(15)`? ¿Qué pasaría si dentro de la función olvidas escribir `return`?

16. Ámbito (scope) básico

El **ámbito** es la zona del programa donde una variable existe y puede usarse. **Analogía:** las habitaciones de una casa; lo que dejas en tu cuarto no lo ven desde la cocina, pero sí puedes mirar hacia el pasillo común. Hay tres zonas que te interesan hoy:

Ámbito	Dónde nace	Quién la ve
Del archivo (global del módulo)	Fuera de toda función y de todo bloque	Todo el archivo, incluidas las funciones
De función	Dentro de `function … { }`	Solo esa función
De bloque	Dentro de cualquier par de llaves `{ }` (`if`, `for`, bloque suelto)	Solo ese bloque (con `let` y `const`)

ejemplo-1.5-bucles-y-funciones.js (ámbito)

```
// ---------- 7. Ámbito (scope) ----------
const tasaIva = 0.19; // ámbito del archivo: visible en todas las funciones

function precioConIva(precio) {
  const resultadoIva = precio * (1 + tasaIva); // ámbito de la función
  return resultadoIva;
}
console.log("Precio con IVA:", precioConIva(10000));

try {
  console.log(resultadoIva); // no existe fuera de la función
} catch (error) {
  console.log(`${error.name}: ${error.message}`);
}

{
  const soloEnBloque = "visible solo entre llaves";
  console.log(soloEnBloque);
}
console.log("Tipo de soloEnBloque afuera:", typeof soloEnBloque);

// Sombreado: una variable interna con el mismo nombre oculta a la externa.
const mensaje = "externo";
function mostrarMensaje() {
  const mensaje = "interno";
  console.log(mensaje);
}
mostrarMensaje();
console.log(mensaje);
```

Salida en la terminal

```
Precio con IVA: 11900
ReferenceError: resultadoIva is not defined
visible solo entre llaves
Tipo de soloEnBloque afuera: undefined
interno
externo
```

- `tasaIva` está en el ámbito del archivo, por eso `precioConIva` puede leerla.

- `resultadoIva` nació dentro de la función; al leerla afuera aparece `ReferenceError: resultadoIva is not defined`. Es un error **útil**: te avisa que usaste una variable donde no existe.
- `soloEnBloque` existe solo entre las llaves; afuera, `typeof` la ve como `undefined`.
- **Sombreado:** si una variable interna tiene el mismo nombre que otra externa, la interna «oculta» a la externa mientras estás dentro. Son variables distintas; evita repetir nombres para no confundirte.

La regla de búsqueda es siempre **de adentro hacia afuera**: JavaScript busca el nombre en el ámbito actual y, si no lo encuentra, sube al siguiente hasta llegar al archivo. Nunca baja hacia ámbitos internos. Por eso conviene declarar cada variable **en el lugar más pequeño donde se necesita** y evitar variables compartidas por todo el archivo.

Pregunta de participación: si declaras `let total = 0;` dentro de un `if` y después de las llaves escribes `console.log(total)`, ¿qué error aparece? ¿Cómo lo resolverías?

17. Entrada del usuario: readline en Node y elementos de la página

Un programa útil necesita recibir datos. En **Node.js** se usa el módulo `node:readline/promises`: muestra una pregunta en la terminal y espera que la persona escriba y pulse Enter. En el **navegador** se leen los campos de la página (`input.value`) cuando ocurre un evento, como el clic en un botón (lo verás en la actividad 1.A5).

saludo-interactivo.js

```
// saludo-interactivo.js · Primer programa interactivo con readline (parte del ejercicio guiado 1.G3)
// Ejecútalo con:  node saludo-interactivo.js

import { createInterface } from "node:readline/promises";
import { stdin, stdout } from "node:process";

// createInterface conecta el teclado (stdin) con la pantalla (stdout).
const rl = createInterface({ input: stdin, output: stdout });

// question muestra la pregunta y espera la respuesta. Siempre devuelve texto (string).
const nombre = await rl.question("¿Cómo te llamas? ");
const edadTexto = await rl.question("¿Cuántos años tienes? ");

const edad = Number(edadTexto); // convertimos el texto a número antes de calcular
console.log(`Hola, ${nombre}. El próximo año tendrás ${edad + 1} años.`);

rl.close(); // libera el teclado; sin esta línea el programa no termina
```

Sesión en la terminal (lo escrito por la persona sigue a cada pregunta)

```
¿Cómo te llamas? Camila
¿Cuántos años tienes? 30
Hola, Camila. El próximo año tendrás 31 años.
```

- `import { createInterface } from "node:readline/promises"` trae una herramienta del propio Node: el prefijo `node:` indica que es un módulo incorporado. No hace falta instalar nada.

- `await rl.question("…")` **espera** la respuesta antes de pasar a la línea siguiente. `await` se explicará a fondo el Día 4; hoy basta con saber que debe acompañar siempre a `rl.question`.
- La respuesta es **siempre texto**: `Number(edadTexto)` la convierte para poder sumar. Si la persona escribe «abc», el resultado es `NaN`, y en tus programas debes comprobarlo con `Number.isNaN`.
- `rl.close()` libera el teclado. Si lo olvidas, el programa se queda esperando y no termina.

En el ejercicio guiado 1.G3 ampliarás este programa hasta construir el juego completo «Adivina el número». La Figura 1.4 muestra una partida real.

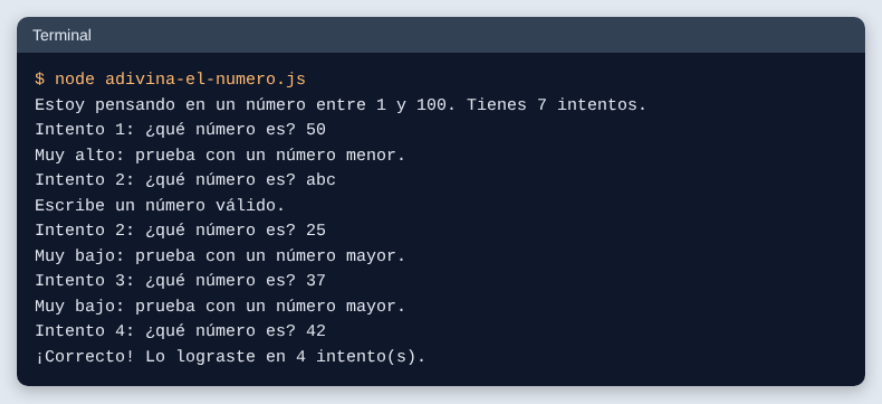


Figura 1.4. Una partida de «Adivina el número» en la terminal (el número secreto era 42).

> **Los programas interactivos se prueban a mano**
> Un script con `readline` espera lo que escribes en el teclado: se ejecuta con `node archivo.js` desde una terminal real y no con el botón de «ejecutar» de algunos editores, que no ofrecen entrada de teclado.

Pregunta de participación: ¿por qué hace falta `Number(...)` antes de comparar la respuesta con el número secreto usando `===`?

18. Leer errores y depurar

Equivocarse es parte de programar: la diferencia entre una persona principiante y una experta no es que la segunda no cometa errores, sino que **sabe leerlos**. Hay cuatro familias que verás todo el taller:

Tipo	Cuándo aparece	Mensaje típico	Ejemplo
Sintaxis	Antes de ejecutar: el código está mal escrito	`SyntaxError: Invalid or unexpected token`	Comillas sin cerrar, falta un `)` o una `}`
Referencia	Al ejecutar: usas un nombre que no existe	`ReferenceError: x is not defined`	Error de ortografía o variable fuera de ámbito
Tipo	Al ejecutar: haces algo que ese valor no permite	`TypeError: Cannot read properties of undefined`	Pedir una propiedad a `undefined` o `null`; reasignar una constante

Tipo	Cuándo aparece	Mensaje típico	Ejemplo
Lógica	El programa corre, pero el resultado es incorrecto	No hay mensaje: aparece `NaN`, `undefined` o un cálculo erróneo	`=` en lugar de `===`, `+` que concatena, olvidar `return`

Cómo leer un error de Node. El mensaje tiene siempre la misma forma: la ruta del archivo y el número de línea, el fragmento de código con una marca `^` debajo del lugar exacto, el tipo de error con su explicación, y la «pila de llamadas» (las líneas que empiezan con `at`). Lee **primero la línea del archivo y la explicación**; la pila suele ser lo menos útil al comienzo.

Mensaje real de Node.js

```
file:///ruta/a/tu/carpeta/a.js:2
console.log(precio * cantidad);
                     ^

ReferenceError: cantidad is not defined
    at file:///ruta/a/tu/carpeta/a.js:2:22
    at ModuleJob.run (node:internal/modules/esm/module_job:343:25)
    at async onImport.tracePromise.__proto__
(node:internal/modules/esm/loader:665:26)
    at async asyncRunEntryPointWithESMLoader
(node:internal/modules/run_main:117:5)
```

Mensaje real de Node.js

```
ANA
LUIS
file:///ruta/a/tu/carpeta/b.js:3
  console.log(nombres[i].toUpperCase());
                         ^

TypeError: Cannot read properties of undefined (reading 'toUpperCase')
    at file:///ruta/a/tu/carpeta/b.js:3:26
    at ModuleJob.run (node:internal/modules/esm/module_job:343:25)
    at async onImport.tracePromise.__proto__
(node:internal/modules/esm/loader:665:26)
    at async asyncRunEntryPointWithESMLoader
(node:internal/modules/run_main:117:5)
```

En el segundo ejemplo el error no está donde estalla: el problema es el `<=` del bucle, que hace una vuelta de más y deja `nombres[2]` en `undefined`. **El mensaje señala dónde se rompió el programa, no siempre dónde nació el error.** Los errores de sintaxis se ven así:

Mensaje real de Node.js

```
file:///ruta/a/tu/carpeta/c.js:1
const nombre = "Café;
               ^^^^^^

SyntaxError: Invalid or unexpected token
    at compileSourceTextModule (node:internal/modules/esm/utils:346:16)
```

Un método de depuración que siempre funciona:

1. **Reproduce** el problema y anota qué esperabas y qué obtuviste.
2. **Lee** el mensaje de error completo y ve a la línea indicada.
3. **Formula una hipótesis**: «creo que `cantidad` llega como texto».

4. **Comprueba** la hipótesis con `console.log(cantidad, typeof cantidad)` o con un punto de interrupción.
5. **Corrige una sola cosa**, guarda, ejecuta de nuevo y observa. Si corriges tres cosas a la vez, no sabrás cuál arregló el problema.

Depurar con la consola. `console.log` con una etiqueta (`console.log("cantidad:", cantidad, typeof cantidad)`) es la herramienta más rápida. Úsala para ver el valor **y el tipo** de lo que sospechas.

Depurar con Sources y debugger. Para detener el programa en una línea y examinar el estado en ese instante:

1. Abre F12 y entra a la pestaña **Sources**. En el panel izquierdo elige tu archivo `.js`.
2. Haz clic sobre el **número de una línea** donde quieras detenerte: aparece un marcador azul (punto de interrupción). También puedes escribir la palabra `debugger;` en el código: con las herramientas abiertas, el navegador se detiene ahí; si no están abiertas, no hace nada.
3. Provoca la acción (por ejemplo, pulsa el botón). La ejecución se **pausa** en esa línea, antes de ejecutarla.
4. En el panel derecho mira **Scope** (todas las variables y sus valores actuales) y agrega expresiones a **Watch** para vigilarlas.
5. Avanza con los botones de la barra o con el teclado: **F10** ejecuta la línea y pasa a la siguiente, **F11** entra a la función que se llama en esa línea y **F8** continúa hasta el siguiente punto de interrupción.
6. Cuando termines, haz clic de nuevo en el marcador para quitarlo y retira cualquier `debugger;` que hayas escrito.

> **Depuración en Node**
> Visual Studio Code incluye un depurador para Node: abre el archivo, coloca un punto de interrupción haciendo clic a la izquierda del número de línea y pulsa **F5**. Funciona igual que Sources: Scope, Watch y los pasos F10 y F11. En el taller lo practicarás en el navegador (actividad 1.A7).

Pregunta de participación: el programa dice «Total: NaN». ¿Qué sospechas primero? ¿Cómo comprobarías tu hipótesis sin cambiar el código?

19. Buenas prácticas del día

Estos hábitos parecen detalles, pero son los que separan un código que se puede mantener de uno que nadie se atreve a tocar. Los aplicarás en todos los ejercicios y se evalúan en la rúbrica:

Práctica	Evita	Prefiere
Nombres descriptivos en camelCase, en español, sin tildes ni ñ	`x`, `dato1`, `añoNacimiento`	`precioUnitario`, `anioNacimiento`
`const` por defecto, `let` solo si cambia, nunca `var`	`var total = 0;`	`const tasaIva = 0.19;` y `let total = 0;`

Práctica	Evita	Prefiere
Comparar con `===` y `!==`	`if (respuesta == 5)`	`if (Number(respuesta) === 5)`
Plantillas de texto para mensajes	`"Total: " + total + " pesos"`	Una plantilla: comillas invertidas con `${total}` dentro del texto
Punto y coma, comillas dobles y sangría de 2 espacios	Estilos mezclados en un mismo archivo	Un solo estilo coherente
Funciones pequeñas, con una tarea y un verbo en el nombre	Un script de 80 líneas sin funciones	`calcularTotal`, `esPar`, `describirClima`
Nombrar los números con significado	`total * 0.19`	`const tasaIva = 0.19;` y `total * tasaIva`
Comentarios que explican el **porqué**	`// suma 1 a i` sobre `i++`	`// el descuento solo aplica los martes`
Validar las entradas de la persona	Calcular con lo que llegue	Comprobar vacío, `NaN` y rangos antes de calcular
Cambiar una cosa, guardar y probar	Corregir cinco cosas y ejecutar	Un cambio por ciclo

> **Convenciones del taller**
> Durante los cinco días todos los archivos de código siguen estas reglas: identificadores en español sin tildes ni ñ y en `camelCase`, `const` por defecto, nunca `var` (salvo el ejemplo que explica por qué), `===` siempre, punto y coma al final de cada instrucción, comillas dobles, sangría de dos espacios y mensajes y comentarios en español.

Ejercicios guiados

Los cuatro ejercicios guiados se hacen en orden y en la carpeta `estudiantes/dia1`. **Escribe el código tú mismo**, sin copiar y pegar: así lo recordarás mejor. Después de cada paso, guarda con `Ctrl + S` y ejecuta. Si te atrasas, pide ayuda al instructor para ponerte al día y no perder el hilo. Las soluciones completas están en `docente/solucionario/dia1`; consúltalas solo al final.

Ejercicio guiado 1.G1 · Ficha de presentación en Node

Tiempo: 15 min | **Entrega:** `ficha-personal.js` en `estudiantes/dia1`.

Objetivo: Declarar variables con const y let, calcular con operadores y mostrar datos con plantillas de texto y typeof.

Instrucciones:

1. Crea el archivo `ficha-personal.js`. Escribe un comentario de cabecera con el nombre del programa y cómo se ejecuta.
2. Declara con `const` tu nombre, tu apellido, tu año de nacimiento, el año actual y tu ciudad (usa tus propios datos, o los del ejemplo). Declara con `let` una variable `tazasDeCafe` con valor 1.
3. Calcula `edad` restando el año de nacimiento al año actual y guárdala en una constante.

4. Muestra la ficha con `console.log` y plantillas: un título, el nombre completo, la edad y la ciudad.
5. Suma 2 a `tazasDeCafe` con el operador `+=` y muéstralo. Calcula `esMayorDeEdad` con `edad >= 18` y muéstralo.
6. Muestra en una sola línea los tipos de `nombre`, `edad` y `esMayorDeEdad` usando `typeof`. Ejecuta con `node ficha-personal.js`.
7. Prueba un error a propósito: escribe `nombre = "Otra persona";` antes de mostrar la ficha, ejecuta y lee el mensaje. Luego bórralo.

Código que debe quedar escrito:

ficha-personal.js

```
// ficha-personal.js · Solución del ejercicio guiado 1.G1
// Ejecútalo con:  node ficha-personal.js

const nombre = "Camila";
const apellido = "Rojas";
const anioNacimiento = 1998;
const anioActual = 2026;
const ciudad = "Medellín";
let tazasDeCafe = 1;

const edad = anioActual - anioNacimiento;

console.log("=== Ficha de presentación ===");
console.log(`Nombre completo: ${nombre} ${apellido}`);
console.log(`Edad: ${edad} años`);
console.log(`Ciudad: ${ciudad}`);

tazasDeCafe += 2;
console.log(`Tazas de café hoy: ${tazasDeCafe}`);

const esMayorDeEdad = edad >= 18;
console.log(`¿Es mayor de edad? ${esMayorDeEdad}`);

console.log("Tipos de dato:", typeof nombre, typeof edad, typeof esMayorDeEdad);
```

Resultado esperado en la terminal:

Salida en la terminal

```
=== Ficha de presentación ===
Nombre completo: Camila Rojas
Edad: 28 años
Ciudad: Medellín
Tazas de café hoy: 3
¿Es mayor de edad? true
Tipos de dato: string number boolean
```

En el paso 7 debe aparecer `TypeError: Assignment to constant variable.`: eso demuestra que `const` protege el valor. Los datos (nombre, ciudad, año) pueden ser distintos; lo importante es que la estructura y los tipos coincidan.

Ejercicio guiado 1.G2 · Conversor de temperatura

Tiempo: 15 min | **Entrega:** `conversor-temperatura.js` en `estudiantes/dia1`.

Objetivo: Escribir funciones con parámetros y retorno, una función flecha, un condicional encadenado y un bucle for.

Instrucciones:

1. Crea `conversor-temperatura.js`. Escribe la función flecha `celsiusAFahrenheit` que reciba `celsius` y devuelva `celsius * 9 / 5 + 32`.
2. Escribe la función declarada `fahrenheitACelsius` que reciba `fahrenheit` y devuelva `(fahrenheit - 32) * 5 / 9`. Prueba ambas con `console.log` y un valor conocido: 100 °C son 212 °F.
3. Escribe `describirClima(celsius)`: devuelve `"frío"` si es menor que 10, `"templado"` si es menor que 25 y `"caluroso"` en los demás casos (usa `if / else if` con `return`).
4. Imprime el encabezado de una tabla y, con un bucle `for` que recorra los grados de 0 a 40 de 10 en 10, una fila por cada valor con Celsius, Fahrenheit y clima. Usa `String(x).padStart(7)` para alinear las columnas.
5. Al final, convierte 98.6 °F a Celsius con `toFixed(1)` y muéstralo con una plantilla.

Código que debe quedar escrito:

conversor-temperatura.js

```
// conversor-temperatura.js · Solución del ejercicio guiado 1.G2
// Ejecútalo con:  node conversor-temperatura.js

// Función flecha de una sola expresión: devuelve el resultado directamente.
const celsiusAFahrenheit = (celsius) => (celsius * 9) / 5 + 32;

// Función declarada con function: usa return de forma explícita.
function fahrenheitACelsius(fahrenheit) {
  return ((fahrenheit - 32) * 5) / 9;
}

function describirClima(celsius) {
  if (celsius < 10) {
    return "frío";
  } else if (celsius < 25) {
    return "templado";
  }
  return "caluroso";
}

console.log("Celsius | Fahrenheit | Clima");
console.log("--------+------------+---------");

for (let c = 0; c <= 40; c += 10) {
  const f = celsiusAFahrenheit(c);
  console.log(`${String(c).padStart(7)} | ${String(f).padStart(10)} |
${describirClima(c)}`);
}

console.log(`La temperatura corporal de 98.6 °F equivale a
${fahrenheitACelsius(98.6).toFixed(1)} °C`);
```

Resultado esperado:

```
Salida en la terminal
Celsius | Fahrenheit | Clima
--------+------------+---------
      0 |         32 | frío
     10 |         50 | templado
     20 |         68 | templado
     30 |         86 | caluroso
     40 |        104 | caluroso
La temperatura corporal de 98.6 °F equivale a 37.0 °C
```

`padStart(n)` rellena un texto con espacios a la izquierda hasta tener `n` caracteres: es un método de los textos que verás con más detalle el Día 2. Observa que `describirClima` devuelve un valor en cada camino; si olvidaras el último `return`, algunas temperaturas mostrarían `undefined`.

Ejercicio guiado 1.G3 · Adivina el número con readline

Tiempo: 20 min | **Entrega:** `saludo-interactivo.js` y `adivina-el-numero.js` en `estudiantes/dia1`.

Objetivo: Combinar entrada del usuario, conversión de tipos, condicionales y un bucle while en un programa interactivo.

Instrucciones:

1. **Calentamiento.** Crea `saludo-interactivo.js` con el código de la sección «Entrada del usuario» (tema 17): importa `createInterface` y `stdin`/`stdout`, haz dos preguntas con `await rl.question(...)`, convierte la edad con `Number` y cierra con `rl.close()`. Ejecútalo con `node saludo-interactivo.js` en una terminal real.
2. Crea `adivina-el-numero.js`. Importa lo mismo que en el calentamiento. Declara `maximo` (100) e `intentosMaximos` (7).
3. Genera el número secreto: `Math.floor(Math.random() * maximo) + 1`. `Math.random()` devuelve un decimal entre 0 y 1; al multiplicarlo por 100, tomar la parte entera y sumar 1 obtienes un entero del 1 al 100. (La línea con `process.env.NUMERO_SECRETO` permite fijar el número desde fuera para hacer pruebas automáticas: puedes incluirla o dejar solo `Math.random`).
4. Declara `let intentos = 0;` y `let acertado = false;`. Escribe el bucle `while (intentos < intentosMaximos && !acertado)`: en cada vuelta pregunta `Intento N: ¿qué número es?`, convierte la respuesta con `Number` y guárdala.
5. Si la respuesta está vacía o `Number.isNaN(numero)` es verdadero, muestra «Escribe un número válido.» y usa `continue` para no gastar un intento.
6. En los demás casos suma 1 a `intentos` y compara con `===`: si es igual, `acertado = true`; si es menor, muestra «Muy bajo»; si no, «Muy alto».
7. Fuera del bucle muestra el mensaje de victoria (con el número de intentos) o el de derrota (con el número secreto). Termina con `rl.close()`.
8. Juega varias partidas. Escribe una letra para comprobar que la validación funciona, y agota los intentos para ver el mensaje de derrota.

Código que debe quedar escrito:

adivina-el-numero.js

```
// adivina-el-numero.js · Solución del ejercicio guiado 1.G3
// Ejecútalo con:  node adivina-el-numero.js

import { createInterface } from "node:readline/promises";
import { stdin, stdout } from "node:process";

const maximo = 100;
const intentosMaximos = 7;

// Para las pruebas automáticas se puede fijar el número con la variable de entorno
NUMERO_SECRETO.
// Si no existe, Number(undefined) es NaN (falsy) y se genera un número al azar.
const secreto = Number(process.env.NUMERO_SECRETO) || Math.floor(Math.random() * maximo) +
1;

const rl = createInterface({ input: stdin, output: stdout });

console.log(`Estoy pensando en un número entre 1 y ${maximo}. Tienes ${intentosMaximos}
intentos.`);

let intentos = 0;
let acertado = false;

while (intentos < intentosMaximos && !acertado) {
  const respuesta = await rl.question(`Intento ${intentos + 1}: ¿qué número es? `);
  const numero = Number(respuesta);

  if (respuesta.trim() === "" || Number.isNaN(numero)) {
    console.log("Escribe un número válido.");
    continue; // no se cuenta como intento
  }

  intentos++;

  if (numero === secreto) {
    acertado = true;
  } else if (numero < secreto) {
    console.log("Muy bajo: prueba con un número mayor.");
  } else {
    console.log("Muy alto: prueba con un número menor.");
  }
}

if (acertado) {
  console.log(`¡Correcto! Lo lograste en ${intentos} intento(s).`);
} else {
  console.log(`Se acabaron los intentos. El número era ${secreto}.`);
}

rl.close();
```

Sesión de ejemplo con el número secreto 42:

```
Sesión en la terminal
Estoy pensando en un número entre 1 y 100. Tienes 7 intentos.
Intento 1: ¿qué número es? 50
Muy alto: prueba con un número menor.
Intento 2: ¿qué número es? abc
Escribe un número válido.
Intento 2: ¿qué número es? 25
Muy bajo: prueba con un número mayor.
Intento 3: ¿qué número es? 37
Muy bajo: prueba con un número mayor.
Intento 4: ¿qué número es? 42
¡Correcto! Lo lograste en 4 intento(s).
```

Fíjate en que el intento «abc» **no gastó** un intento: el número vuelve a decir «Intento 2». Eso lo consigue `continue`, que salta antes de `intentos++`.

Ejercicio guiado 1.G4 · Contador de clics en la plantilla inicial

Tiempo: 20 min | **Entrega:** Carpeta `contador` (copia de la plantilla) con la página funcionando al abrirla con doble clic.

Objetivo: Conectar un archivo JavaScript con una página usando defer, guardar un estado en una variable y reflejarlo en la pantalla y en la consola.

Instrucciones:

1. Copia la carpeta `estudiantes/00_plantilla_inicial` en la misma carpeta `estudiantes` y llámala `contador` (debe quedar al lado de la original para que la ruta `../recursos/estilos-base.css` siga funcionando).
2. Abre `contador/index.html`. Cambia el título y el `<h1>`. Dentro de `<main>`, reemplaza el contenido de la sección por el HTML que se muestra abajo: un valor actual dentro de `<strong id="valor">` y tres botones con los `id` `boton-sumar`, `boton-restar` y `boton-reiniciar`.
3. Abre `contador/app.js` y borra el contenido de ejemplo. Declara `let contador = 0;` y busca los cuatro elementos con `document.getElementById`.
4. Escribe la función `actualizarPantalla()`: copia el contador en el texto del elemento (`valor.textContent = String(contador);`) y lo muestra en la consola con una plantilla.
5. Con `addEventListener("click", () => { … })` programa cada botón: sumar incrementa; restar disminuye **solo si** el contador es mayor que 0 (en caso contrario muestra un aviso en el párrafo `aviso-limite`); reiniciar vuelve a 0. Cada manejador termina llamando a `actualizarPantalla()`.
6. Abre `index.html` con doble clic y la consola con F12. Pulsa los botones y observa la página y los mensajes. Recarga con F5: el contador vuelve a 0 (los datos se pierden al recargar; el Día 3 verás cómo conservarlos).

HTML de la sección (dentro de main):

contador/index.html (extracto)

```
  <main class="contenedor">
    <section>
      <h2>Tazas servidas hoy</h2>
      <p>Pulsa los botones para cambiar el contador. Mira también la consola (F12).</p>

      <p>Valor actual: <strong id="valor" role="status" aria-live="polite">0</strong></p>

      <p>
        <button type="button" class="boton" id="boton-sumar">Sumar 1</button>
        <button type="button" class="boton boton--secundario" id="boton-restar">Restar
1</button>
        <button type="button" class="boton boton--peligro" id="boton-
reiniciar">Reiniciar</button>
      </p>
      <p id="aviso-limite" class="campo__error"></p>
    </section>
```

JavaScript que debe quedar escrito:

contador/app.js

```
// contador-clics.js · Solución del ejercicio guiado 1.G4
// Se carga con <script src="contador-clics.js" defer>: el HTML ya existe cuando se
ejecuta.

let contador = 0;

const valor = document.getElementById("valor");
const botonSumar = document.getElementById("boton-sumar");
const botonRestar = document.getElementById("boton-restar");
const botonReiniciar = document.getElementById("boton-reiniciar");
const avisoLimite = document.getElementById("aviso-limite");

// Una sola función se encarga de reflejar el dato en la pantalla.
function actualizarPantalla() {
  valor.textContent = String(contador);
  console.log(`Contador: ${contador}`);
}

botonSumar.addEventListener("click", () => {
  contador++;
  avisoLimite.textContent = "";
  actualizarPantalla();
});

botonRestar.addEventListener("click", () => {
  if (contador > 0) {
    contador--;
    avisoLimite.textContent = "";
  } else {
    avisoLimite.textContent = "El contador no puede ser negativo.";
  }
  actualizarPantalla();
});

botonReiniciar.addEventListener("click", () => {
  contador = 0;
  avisoLimite.textContent = "";
  actualizarPantalla();
});
```

Qué debes ver: un valor actual que cambia al pulsar «Sumar 1» y «Restar 1»; al intentar restar con el contador en 0 aparece en rojo «El contador no puede ser negativo.», y en la consola aparece

una línea `Contador: n` por cada acción. Nota: en la solución del docente la ruta del CSS es distinta, porque el archivo está en otra carpeta; en tu copia mantén el `<link>` que ya trae la plantilla.

Actividades individuales

Son siete actividades de dificultad creciente. Las dos primeras exploran los entornos y los tipos; las siguientes construyen programas pequeños; la 1.A6 y la 1.A7 son de **detección y corrección de errores** y de **depuración**. Guarda cada archivo con el nombre indicado en la carpeta `estudiantes/dia1` (o en su carpeta de proyecto).

Actividad individual 1.A1 · Explora la consola de Node y la del navegador

Tiempo: 15 min | **Nivel:** Básico | **Entrega:** Una hoja (o un comentario en un archivo) con los resultados anotados y una conclusión de dos frases.

Objetivo: Comparar los dos entornos donde corre JavaScript y reconocer qué ofrece cada uno.

Instrucciones:

1. Ejecuta `node ejemplo-1.1-hola-node.js`. Cambia el mensaje de bienvenida por el tuyo, guarda y vuelve a ejecutar.
2. Abre `ejemplo-1.2-hola-navegador.html` con doble clic. Pulsa F12, entra a **Console**, pulsa tres veces «Saludar» y anota las líneas que aparecen.
3. En la consola del navegador, escribe una a una estas expresiones y anota el resultado de cada una: `typeof "hola"`, `2 ** 10`, `"5" + 3`, `"5" - 3` y `document.title`.
4. En una terminal, escribe `node` (modo interactivo) y repite las cuatro primeras expresiones. Luego escribe `document.title` y anota qué ocurre. Sal con `.exit`.
5. Escribe en dos frases qué diferencia hay entre la consola del navegador y la de Node.js, y por qué crees que ocurre.

Criterios de logro: modificaste y ejecutaste el ejemplo 1.1; anotaste las cinco líneas de la consola del navegador (dos de arranque y tres de clics, contando el título); los resultados de las cuatro primeras expresiones coinciden en ambos entornos; y tu conclusión menciona que `document` existe solo en el navegador.

Actividad individual 1.A2 · Laboratorio de tipos y comparaciones

Tiempo: 10 min | **Nivel:** Básico | **Entrega:** `laboratorio-tipos.js` y una columna de predicciones con los aciertos marcados.

Objetivo: Predecir el resultado de expresiones con tipos y conversiones, y contrastar la predicción con la ejecución real.

Instrucciones:

1. **Antes de escribir código**, anota en tu cuaderno qué crees que resulta de cada expresión de la tabla.
2. Crea `laboratorio-tipos.js`. Escribe una función `mostrar(expresion, resultado)` que imprima `expresion.padEnd(28)`, una flecha `->` y el resultado.

3. Llama a `mostrar` una vez por cada expresión, escribiendo la expresión como texto y como código (por ejemplo, `mostrar("5 == \"5\"", 5 == "5")`).
4. Ejecuta el archivo y marca con ✓ o ✗ cada predicción. Elige las **dos** expresiones que más te sorprendieron y explica en un comentario por qué da ese resultado.

N.º	Expresión	N.º	Expresión
1	`typeof null`	8	`Number("abc")`
2	`"5" + 3`	9	`parseInt("15.9")`
3	`"5" - 3`	10	`Boolean("false")`
4	`"5" * "2"`	11	`5 == "5"`
5	`"hola" * 2`	12	`5 === "5"`
6	`true + 1`	13	`null == undefined`
7	`Number("")`	14	`0.1 + 0.2 === 0.3`

Criterios de logro: el archivo se ejecuta sin errores y muestra las 14 expresiones; marcaste tus aciertos; y las dos explicaciones mencionan conversión implícita o la diferencia entre `==` y `===`.

Actividad individual 1.A3 · Tabla de multiplicar y otros bucles

Tiempo: 10 min | **Nivel:** Básico | **Entrega:** `tabla-multiplicar.js` con las cuatro partes ejecutadas.

Objetivo: Practicar for, while, break y bucles anidados con una variable de control bien definida.

Instrucciones:

1. **Parte 1.** Declara `const numero = 7;` y, con un `for` de 1 a 10, imprime `7 x 1 = 7`, `7 x 2 = 14`… con plantillas. Cambia `numero` y comprueba que la tabla cambia.
2. **Parte 2.** Con un `while`, calcula la suma de los números pares del 1 al 20 (usa `%` para reconocer los pares) y muéstrala.
3. **Parte 3.** Con dos bucles `for` anidados, imprime una cuadrícula de multiplicación de 5 filas por 5 columnas (cada fila en una sola línea; construye el texto de la fila con `let linea = ""` y `+=`).
4. **Parte 4.** Con un `for` descendente, imprime una cuenta regresiva de 5 a 1 y, al llegar a 0, muestra «¡Despegue!» y usa `break`.

Criterios de logro: las cuatro partes producen los resultados correctos (la suma de pares da 110); no hay bucles infinitos; y cada variable de control está declarada con `let` dentro del propio `for` o antes del `while`.

Actividad individual 1.A4 · Validador de contraseña

Tiempo: 15 min | **Nivel:** Intermedio | **Entrega:** `validador-contrasena.js` con la función `validarContrasena` y la prueba con cinco contraseñas.

Objetivo: Diseñar una función que devuelve un resultado a partir de varias reglas, combinando condicionales, un bucle for...of y banderas booleanas.

Instrucciones:

1. Define las reglas: mínimo 8 caracteres, al menos un dígito, al menos una mayúscula y ningún espacio.
2. Escribe la función `validarContrasena(contrasena)` que devuelva una **lista de problemas** (vacía si la contraseña es válida). Empieza con `const problemas = [];` y añade cada mensaje con `problemas.push("…")`. `push` agrega un elemento al final de la lista: se verá el Día 2.
3. Para la longitud usa `contrasena.length`. Para los dígitos y las mayúsculas, recorre la contraseña con `for (const letra of contrasena)` y usa dos banderas (`tieneDigito`, `tieneMayuscula`). Pista: `"0123456789".includes(letra)` dice si la letra es un dígito, y una letra es mayúscula si `letra !== letra.toLowerCase()`.
4. Para los espacios usa `contrasena.includes(" ")`. Devuelve la lista al final de la función.
5. Prueba con cinco candidatas (`"abc"`, `"contrasena1"`, `"Contrasena"`, `"Con tra 1A"` y `"Segura2026"`): para cada una imprime «válida» o «NO válida» con los problemas unidos con `problemas.join("; ")`.

Criterios de logro: la función no imprime nada (solo devuelve la lista); cada contraseña de prueba muestra exactamente los problemas que tiene; y `"Segura2026"` es la única válida.

Actividad individual 1.A5 · Calculadora de propina en el navegador

Tiempo: 20 min | **Nivel:** Intermedio | **Entrega:** Carpeta `propina` (copia de la plantilla) que funciona al abrir `index.html` con doble clic.

Objetivo: Leer datos de campos de una página, convertirlos y validarlos, y mostrar el resultado en pantalla.

Instrucciones:

1. Copia `00_plantilla_inicial` como `propina` (al lado de la original). En `index.html` reemplaza el contenido de `<main>` por un campo numérico `cuenta` (con su `<label>`), un `<select>` `porcentaje` con las opciones 0, 10, 15 y 20, un botón `boton-calcular`, un párrafo `error-cuenta` con `role="alert"` y un párrafo `resultado` con `role="status"`.
2. En `app.js`, busca los elementos con `getElementById` y crea el formateador de pesos: `new Intl.NumberFormat("es-CO", { style: "currency", currency: "COP", maximumFractionDigits: 0 })`.
3. Escribe la función `calcularPropina(cuenta, porcentaje)` que devuelva `cuenta * porcentaje / 100`.

4. En el manejador del clic: lee `campoCuenta.value` (¡es texto!), conviértelo con `Number` y convierte también el porcentaje. Si el texto está vacío (`trim() === ""`), el número es `NaN` o es menor o igual que 0, muestra «Escribe un valor de cuenta mayor que cero.» en `error-cuenta`, limpia el resultado y termina con `return`.
5. Si los datos son válidos, limpia el error y calcula la propina y el total. Muestra «Propina: … · Total a pagar: …» en `resultado`, con los valores formateados como pesos. Escribe también una línea en la consola con los tres valores.
6. Prueba con 50.000 y 15 % (debe dar 7.500 y 57.500), con el campo vacío y con un valor negativo.

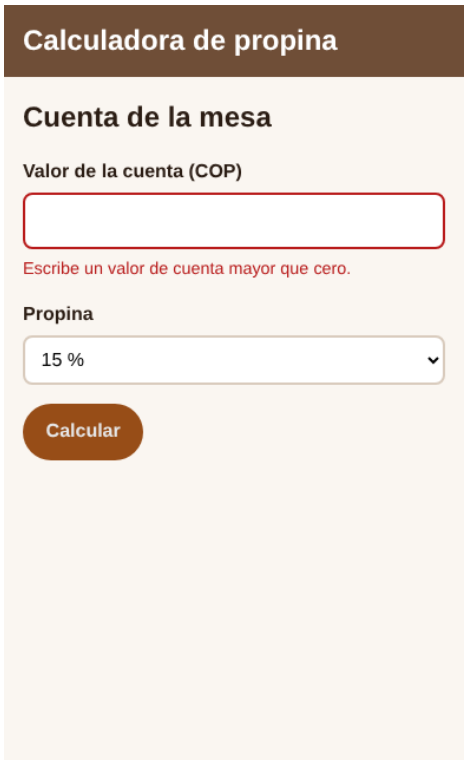


Figura 1.5. La calculadora en un teléfono con un dato inválido: el mensaje de error aparece bajo el campo.

Criterios de logro: el resultado de 50.000 con 15 % es «Propina: $ 7.500 · Total a pagar: $ 57.500»; los tres casos inválidos muestran el mensaje; el código convierte con `Number` antes de calcular; y no hay `var`, `==` ni `innerHTML`.

Actividad individual 1.A6 · Detective de errores: la cuenta de la mesa (ejercicio 1.6)

Tiempo: 20 min | **Nivel:** Intermedio-avanzado | **Entrega:** `ejercicio-1.6-corregido.js` y una tabla con los errores encontrados (línea original, tipo, síntoma y corrección).

Objetivo: Detectar y corregir errores de sintaxis, de ejecución y de lógica leyendo los mensajes de Node y comprobando los resultados.

Instrucciones:

1. Copia `ejercicio-1.6-codigo-con-errores.js` con el nombre `ejercicio-1.6-corregido.js` y léelo completo. El encabezado indica la salida que debe producir cuando esté correcto.
2. Ejecútalo con `node`. El archivo contiene **doce errores**: tres de sintaxis (Node los muestra de uno en uno), tres que detienen el programa durante la ejecución y seis de lógica que producen resultados incorrectos.
3. Trabaja con el método del tema 18: lee el mensaje, ve a la línea, formula una hipótesis, corrige **una sola cosa**, guarda y ejecuta de nuevo.
4. Por cada error encontrado agrega una fila a tu tabla: línea del archivo original, tipo (sintaxis, referencia, tipo o lógica), síntoma o mensaje de Node y la corrección.
5. Cuando el programa deje de mostrar errores, compara **línea por línea** su salida con la salida esperada del encabezado: los errores de lógica no producen mensaje, solo resultados distintos.

Criterios de logro: el programa produce exactamente la salida del encabezado; la tabla tiene los doce errores con su tipo; no cambiaste los datos de entrada (precios, cantidades ni tasas) para «acomodar» el resultado; y escribiste con tus palabras por qué se produjo cada error de lógica.

Actividad individual 1.A7 · Depurar el promedio con Sources y debugger (ejercicio 1.7)

Tiempo: 10 min | **Nivel:** Intermedio-avanzado | **Entrega:** `ejercicio-1.7-corregido.js` (con su HTML apuntando a él) y dos frases que expliquen cada error.

Objetivo: Usar puntos de interrupción y el panel Scope para localizar errores de lógica que no producen mensajes.

Instrucciones:

1. Abre `ejercicio-1.7-depurar-promedio.html` con doble clic y pulsa «Calcular». El resultado correcto es **promedio 3.86, 4 notas aprobadas de 5 y mejor nota 5**. Anota qué valores aparecen en realidad (Figura 1.6).
2. Pulsa F12 y entra a **Sources**. Abre `ejercicio-1.7-depurar-promedio.js` y haz clic en el número de la línea `suma += lista[i];` para poner un punto de interrupción. Pulsa «Calcular».
3. Con la ejecución pausada, mira el panel **Scope**: ¿cuánto vale `i` en la primera pausa? Avanza con F8 y observa cómo cambian `i` y `suma`. ¿Qué nota se sumó primero? ¿Cuál no se sumó nunca?

4. Quita ese punto de interrupción y coloca otro dentro de `contarAprobadas`, en la línea del `if`. Cuando `nota` valga 3.0, evalúa en **Watch** la expresión `nota > minima`. ¿Debería esa nota aprobar? Explica el resultado.
5. Corrige los dos errores en una **copia** del archivo (`ejercicio-1.7-corregido.js`; actualiza el `src` del HTML), recarga y comprueba que los tres resultados son correctos. Escribe dos frases explicando cada error.

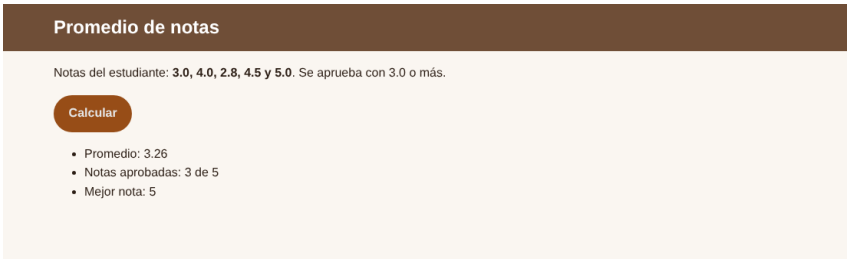


Figura 1.6. El ejercicio 1.7 con los errores sin corregir: el promedio y la cantidad de aprobadas son incorrectos.

Criterios de logro: localizaste los dos errores con puntos de interrupción (no por simple lectura); la versión corregida muestra 3.86, «4 de 5» y 5; y las dos explicaciones nombran la causa exacta (índice inicial del bucle y operador de comparación).

Retos de profundización

Trabaja en archivos nuevos para no alterar los entregables del día. El reto 1.R3 es opcional: hazlo si terminas antes.

Reto 1.R1 · Adivina el número, versión 2

Tiempo: 15 min | **Nivel:** Avanzado | **Entrega:** `adivina-el-numero-v2.js` con una partida completa y al menos dos niveles de dificultad.

Objetivo: Estructurar un programa interactivo en funciones y añadir niveles, pistas de temperatura y la opción de jugar otra vez.

Instrucciones:

1. Parte de tu `adivina-el-numero.js`. Antes de jugar, pregunta el **nivel**: 1 (del 1 al 10, 5 intentos), 2 (del 1 al 50, 7 intentos) o 3 (del 1 al 100, 7 intentos). Resuélvelo con un `switch` que asigne `maximo` e `intentosMaximos`.
2. Mueve la lógica de una partida a la función `jugarPartida(maximo, intentosMaximos)` (declarada con `async function`, para poder usar `await` dentro) que devuelva `true` si la persona acertó y `false` si no.
3. Escribe la función `darPista(distancia)`: devuelve «¡Caliente!» si la distancia al número secreto es 3 o menos, «Tibio» si es 10 o menos y «Frío» en los demás casos. Calcula la distancia con `Math.abs(numero - secreto)`.
4. Al terminar cada partida pregunta «¿Jugar otra vez? (s/n)» y repite mientras la respuesta sea `s` (acepta mayúscula o minúscula con `toLowerCase()`).
5. Al final muestra cuántas partidas ganó y cuántas jugó.

Reto 1.R2 · Cuenta de Café Aroma por consola

Tiempo: 15 min | **Nivel:** Avanzado | **Entrega:** `cuenta-cafe-aroma.js` que imprime una cuenta ordenada con subtotal, descuento, IVA, propina y total.

Objetivo: Integrar variables, funciones, switch y formato de salida en un programa que se parezca a un caso real.

Instrucciones:

1. Declara tres productos con su nombre, precio y cantidad en constantes separadas (el Día 2 los guardarás en arreglos de objetos): por ejemplo 2 capuchinos a 9.500, 1 croissant a 5.500 y 2 tortas de zanahoria a 8.500.
2. Escribe `calcularLinea(precio, cantidad)` y calcula el subtotal.
3. Escribe `obtenerPorcentajeDescuento(dia)` con un `switch`: el «martes de café» da 15 %, el domingo 5 % y cualquier otro día 0 %. Declara una constante con el día de la semana.
4. Calcula descuento, base gravable (subtotal menos descuento), IVA del 19 % y propina del 10 %, ambos sobre la base gravable, y el total.
5. Imprime la cuenta alineada: usa `padEnd` y `padStart` para las columnas, `"-".repeat(40)` para las líneas y `Intl.NumberFormat` para los pesos.

Reto 1.R3 · FizzBuzz y números primos (opcional)

Tiempo: Opcional | **Nivel:** Avanzado | **Entrega:** `fizzbuzz-primos.js` con las dos partes.

Objetivo: Resolver dos problemas clásicos que combinan residuo, condicionales y bucles anidados.

Instrucciones:

1. **FizzBuzz.** Escribe `fizzBuzz(numero)`: devuelve «FizzBuzz» si el número es múltiplo de 3 y de 5, «Fizz» si es múltiplo de 3, «Buzz» si es múltiplo de 5 y, en los demás casos, el propio número como texto. Imprime los resultados del 1 al 20. ¿En qué orden debes escribir las condiciones y por qué?
2. **Primos.** Escribe `esPrimo(numero)`: un número es primo si es mayor que 1 y no tiene divisores entre 2 y su raíz cuadrada. Pista: el bucle `for (let divisor = 2; divisor * divisor <= numero; divisor++)` evita calcular la raíz.
3. Recorre los números del 1 al 50, junta los primos en un texto separado por espacios y muestra cuántos hay (son 15).

Preguntas de análisis

Responde de forma individual, sin mirar tus apuntes. Después podrás revisar las respuestas con el instructor. Hay preguntas de opción múltiple, de verdadero o falso, abiertas y de lectura o corrección de código.

1. ¿Cuál es la relación correcta entre JavaScript y ECMAScript?

- **A)** ECMAScript es una biblioteca de funciones que se instala con npm y que JavaScript necesita para funcionar.
- **B)** ECMAScript es la especificación (el estándar) que define el lenguaje; JavaScript es su implementación más conocida.
- **C)** JavaScript se usa solo en el navegador y ECMAScript solo en Node.js.
- **D)** ECMAScript es la versión antigua del lenguaje que fue reemplazada por JavaScript.

2. ¿Para qué sirve el atributo `defer` en `<script src="app.js" defer></script>` colocado dentro de `<head>`?

- **A)** Hace que el script se ejecute dos veces para comprobar que no tiene errores.
- **B)** Retrasa la ejecución del script hasta que el navegador terminó de leer el HTML, de modo que los elementos de la página ya existen.
- **C)** Impide que otras personas lean el código fuente del script.
- **D)** Permite usar `import` y `export` dentro del archivo.

3. ¿Cuál es el resultado de `"7" + 3 - 2`?

- **A)** 8
- **B)** 71
- **C)** "732"
- **D)** NaN

4. ¿Cuál de estos valores es **truthy** (se comporta como verdadero en un `if`)?

- **A)** `0`
- **B)** `""` (cadena vacía)
- **C)** `"0"` (el texto cero)
- **D)** `NaN`

5. ¿Cuántas veces se ejecuta el cuerpo de este bucle? `for (let i = 2; i < 10; i += 3) { … }`

- **A)** 2 veces
- **B)** 3 veces
- **C)** 4 veces
- **D)** 8 veces

6. ¿Qué devuelve `typeof null`?

- **A)** `"null"`
- **B)** `"undefined"`
- **C)** `"object"`
- **D)** `"boolean"`

7. El operador `==` compara el valor y el tipo, por lo que `5 == "5"` da `false`.

( ) Verdadero ( ) Falso Justifica: _____

8. Una función que no ejecuta ninguna instrucción `return` devuelve `undefined`.

( ) Verdadero ( ) Falso Justifica: _____

9. Una variable declarada con `let` dentro de un bloque `if { … }` puede leerse después de la llave de cierre del bloque.

( ) Verdadero ( ) Falso Justifica: _____

10. Todo lo que una persona escribe con `readline` o en un campo de formulario llega como texto (`string`), aunque sean solo dígitos.

( ) Verdadero ( ) Falso Justifica: _____

11. Explica con tus palabras la diferencia entre `null` y `undefined` y menciona una situación en la que aparece cada uno.

Respuesta: _____

12. Explica por qué se recomienda declarar con `const` por defecto y usar `let` solo cuando el valor va a cambiar.

Respuesta: _____

13. Describe los pasos para detener un programa en una línea con un punto de interrupción en la pestaña Sources y explica qué ventaja tiene frente a poner varios `console.log`.

Respuesta: _____

14. Lee el código y escribe exactamente lo que imprime en la consola. Explica el papel de `continue`.

pregunta.js

```
let total = 0;
for (let i = 1; i <= 4; i++) {
  if (i === 3) {
    continue;
  }
  total += i;
}
console.log(total);
```

Respuesta: _____

15. Al ejecutar este programa se imprime `Resultado: undefined` en lugar de `Resultado: menor`. Explica por qué y corrige la función.

pregunta.js

```
function clasificarEdad(edad) {
  if (edad >= 18) {
    return "mayor";
  }
}

const mensaje = "Resultado: " + clasificarEdad(15);
console.log(mensaje);
```

Respuesta: _____

16. La persona escribe 5 y el programa responde «Sigue intentando» aunque el número secreto es 5. Encuentra la causa y corrige el código.

```
pregunta.js
const respuesta = "5"; // lo que escribió la persona con readline
const secreto = 5;

if (respuesta === secreto) {
  console.log("¡Acertaste!");
} else {
  console.log("Sigue intentando");
}
```

Respuesta: _____

17. Escribe lo que imprimen las dos líneas finales y explica el fenómeno de ámbito que ocurre.

```
pregunta.js
const nombre = "global";

function mostrar() {
  const nombre = "local";
  return nombre;
}

console.log(mostrar());
console.log(nombre);
```

Respuesta: _____

Entregable del día

Entregable del día: Primeros programas en JavaScript

- `ficha-personal.js`, `conversor-temperatura.js`, `saludo-interactivo.js` y `adivina-el-numero.js`, que se ejecutan con `node` sin errores.
- La carpeta `contador` (ejercicio 1.G4) y la carpeta `propina` (actividad 1.A5), que funcionan al abrir su `index.html` con doble clic y muestran mensajes en la consola.
- `laboratorio-tipos.js`, `tabla-multiplicar.js` y `validador-contrasena.js` (actividades 1.A2 a 1.A4).
- `ejercicio-1.6-corregido.js` con su tabla de doce errores, y `ejercicio-1.7-corregido.js` con las dos frases explicativas.
- Opcional: `adivina-el-numero-v2.js`, `cuenta-cafe-aroma.js` y `fizzbuzz-primos.js` de los retos.
- Todo el código usa `const` por defecto, `===`, plantillas de texto, comillas dobles, punto y coma y nombres en español sin tildes.

Lista de verificación del día

- ☐ Puedo ejecutar un archivo con `node archivo.js` desde la carpeta correcta y abrir la consola del navegador con F12.
- ☐ Mis páginas enlazan el script con `<script src="…" defer>` y no veo errores en la consola.

- [ ] Declaré todo con `const` y usé `let` solo donde el valor cambia; no usé `var`.
- [ ] Convierto con `Number(...)` los datos que llegan como texto antes de calcular o comparar.
- [ ] Comparo siempre con `===` y `!==`, y sé explicar la diferencia con `==`.
- [ ] Mis mensajes usan plantillas de texto y no concatenaciones con `+`.
- [ ] Mis funciones hacen una sola tarea, tienen nombre con verbo y devuelven su resultado con `return`.
- [ ] Sé en qué ámbito vive cada variable y no dependo de variables declaradas dentro de otro bloque.
- [ ] Encontré y corregí los doce errores del ejercicio 1.6 explicando el tipo de cada uno.
- [ ] Usé un punto de interrupción en Sources para localizar un error de lógica (ejercicio 1.7).
- [ ] Los programas validan las entradas inválidas (vacío, texto en lugar de número, valores negativos).

Rúbrica de evaluación del día

Rúbrica del Día 1: Fundamentos de JavaScript (10 puntos)

Criterio	Excelente (100 %)	Satisfactorio (75 %)	Básico (50 %)	Insuficiente (0–25 %)	Puntos
Entorno y ejecución	Ejecuta scripts en Node y páginas con defer sin ayuda; usa terminal y consola con soltura.	Ejecuta Node y páginas con defer; necesita ayuda ocasional con rutas o comandos.	Ejecuta solo con guía paso a paso; el script no se carga sin ayuda.	No logra ejecutar programas en Node ni cargar el script en la página.	**1**
Variables, tipos y conversión	const por defecto, nombres claros, convierte con Number y compara con === siempre.	Usa const y let bien; algún nombre poco claro o una comparación con ==.	Mezcla var, let y const; olvida convertir tipos en alguna entrada.	Usa var o nombres sin sentido; sus cálculos con texto dan resultados erróneos.	**2**
Condicionales y bucles	Elige if, switch o ternario con criterio; los bucles terminan y cubren los casos límite.	Condicionales y bucles correctos; falla un caso límite o repite código.	Resuelve solo con ayuda; hay errores por uno o ramas sin cubrir.	Las decisiones y los bucles no producen el resultado pedido o no terminan.	**2**
Funciones y ámbito	Funciones pequeñas con parámetros y return; ninguna variable depende de otro ámbito.	Funciones correctas con return; alguna hace más de una tarea.	Funciones sin return o con variables compartidas innecesarias.	No separa el código en funciones o genera errores de ámbito.	**2**
Interacción: readline y página	Lee con readline y con campos de página, valida entradas y muestra mensajes claros.	Lee y muestra datos correctamente; valida solo algunos casos inválidos.	Lee datos pero no los convierte ni valida; el resultado falla con entradas raras.	El programa interactivo o la página no responden a la entrada.	**1**